- **直线、圆弧的生成算法**
  - DDA直线（微分法）
  - 中点画线法
    - 浮点型
    - 优化后的整数型
  - Bresenham直线
    - 浮点型
    - 整数型
  - Bresenham画圆
  - 中点画圆法
  - （线宽生成）刷子绘制法
    - 线刷子绘制法
    - 方形刷子绘制法
  - 实区域填充法
  - 线型的处理
- **多边形填充算法**
  - 种子填充法
  - 扫描线种子填充法
  - 扫描线填充算法
    - 边表ET
    - 活动边表AET
- **图形反走样技术**
  - 像素细分技术
  - Bresenham反走样技术
- **曲线、曲面**
  - 贝塞尔Bezier曲线(逼近控制点)
  - B样条曲线
    - 二次B样条曲线的矩阵表示
    - 三次B样条曲线的矩阵表示
  - Bezier曲面
  - B样条曲面
- **图形变换**
  - 二维图形变换
    - 平移、旋转
    - 放缩
  - 三维变换
    - 平移
    - 比例
    - 绕x轴逆时针旋转
    - 绕y轴逆时针旋转
    - 绕z轴逆时针旋转
  - 坐标系变换
- **裁剪算法**
  - Cohen-Sutherland直线裁剪算法（编码）
  - Lang-Barsky直线裁剪算法（直线参数方程）
  - Sutherland-Hodgeman多边形裁剪算法
  - Weiler-Atherton多边形裁剪算法
  - 长方体裁剪算法

视椎体裁剪算法
**图形消隐**
背面剔除算法
画家算法
Weiler-Atherton算法
BSP树算法
深度缓冲区算法
算法步骤
Warnock算法
**模型生成**
地形生成算法
根据高度图生成高度
Diamond-Square算法
雨雪模拟算法
**基本概念**
什么是计算机图形学
研究内容
图形和图像的区别
显卡工作流程
显存
GPU
显卡分辨率
渲染流水线
光照的基本原理
光照模型、光源
纹理
三维模型顶点的数据结构表示
模型的表示方法
Direct3D 12图形绘制的基本原理
3D实体绘制步骤
期末题目回忆
简答题:
计算题
画图题
分析题
算法题
**笔记作者：swjtu**

# 直线、圆弧的生成算法

## DDA直线（微分法）

步进主方向加斜率（y = y + k或x = x + 1/k），每个点都四舍五入

### 当|K| <= 1 时

x轴方向为步 进的主方向

> - X(i+1) = Xi + 1;
> - Y(i+1) = Yi + K;
>
> **备注**：因为像素点坐标是整数，所有Y值要四舍五入，即int(Y+0.5)

### 当|K| > 1 时

y轴方向为步进的主方向

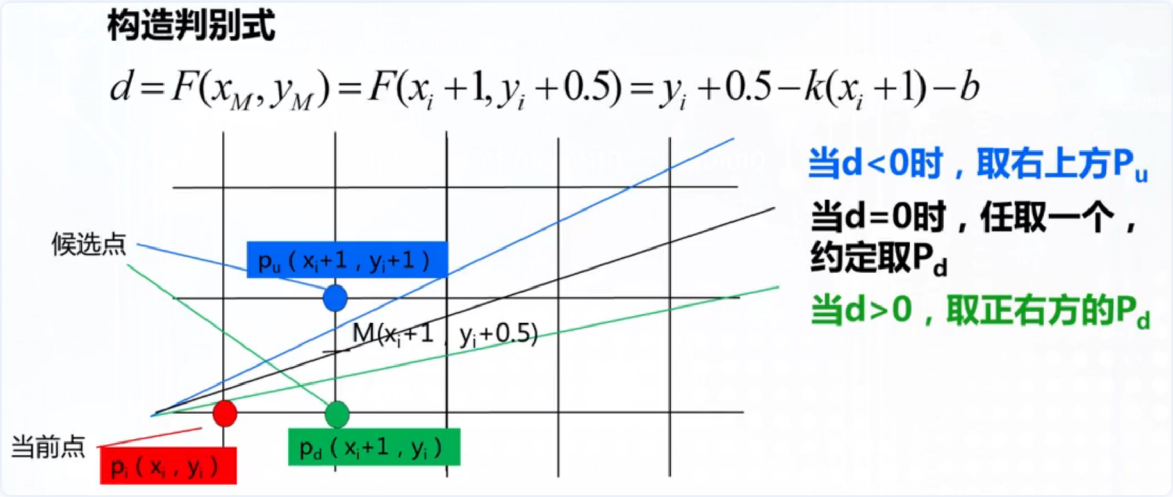
> - Y(i+1) = Yi + 1;
> - X(i+1) = Xi + 1/k;
>
> **备注**：因为像素点坐标是整数，所有X值要四舍五入，即int(X+0.5)

## 中点画线法

### 判断点(x+1, y+0.5)在直线下方还是上方

> 以下生成算法都建立于 `0<k<1` 的基础上，也就是x方向为主步长
>
> 如果 `k>1` ，就把算式中x和y的位置互换，k换成1/k

## 浮点型




- d < 0，中点在直线下方，下一个点取(x + 1, y + 1)
- d >= 0，中点在直线上方，下一个点取(x + 1, y)

$d_i$决定$y_{i+1}$的取值

**d的初值**

$$\begin{aligned} d_0 &= F(x_0+1, y_0+0.5) \\ &= y_0+0.5-k(x_0+1)-b \\ &= y_0-kx_0-b-k+0.5 \\ &= 0.5-k \end{aligned}$$

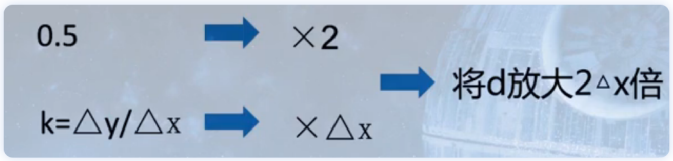
**d < 0**

$$\begin{aligned} d &= F(x_i+2, y_i+1.5) \\ &= y_i+1.5-k(x_i+2)-b \\ &= y_i+1.5-k(x_i+1)-b-k \\ &= d+1-k \end{aligned}$$

**d ≥ 0**

$$\begin{aligned} d &= F(x_i+2, y_i+0.5) \\ &= y_i+0.5-k(x_i+2)-b \\ &= y_i+0.5-k(x_i+1)-b-k \\ &= d-k \end{aligned}$$

## 优化后的整数型



$$d_0 = \Delta x - 2\Delta y$$

> $\triangle y = y2 - y1$
> $\triangle x = x2 - x1$
> 笔记中的公式基于这个变化关系

- 如果d < 0

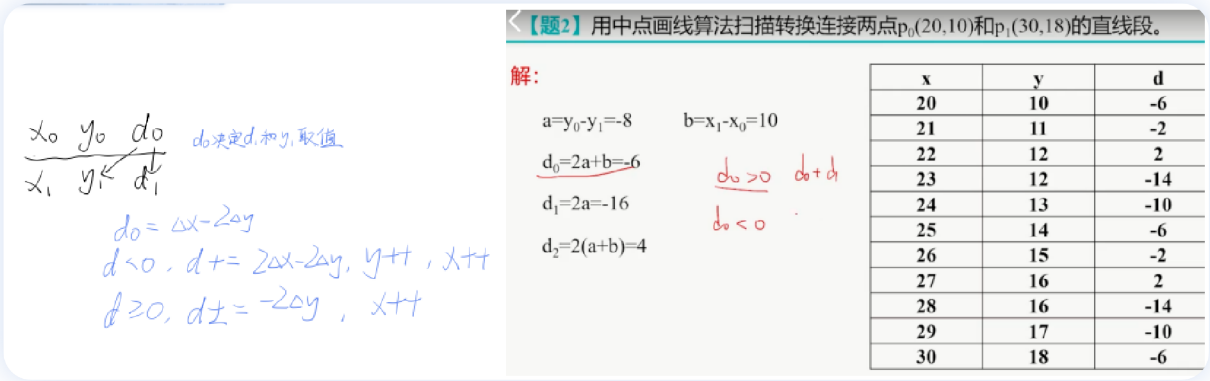
  (x, y) 更新为 (x + 1, y + 1)

  $$d = d + 2\Delta x - 2\Delta y$$

- 如果d >= 0

  (x, y) 更新为 (x + 1, y)

  $$d = d - 2\Delta y$$




| x | y | d |
|---|---|---|
| 20 | 10 | -6 |
| 21 | 11 | -2 |
| 22 | 12 | 2 |
| 23 | 12 | -14 |
| 24 | 13 | -10 |
| 25 | 14 | -6 |
| 26 | 15 | -2 |
| 27 | 16 | 2 |
| 28 | 16 | -14 |
| 29 | 17 | -10 |
| 30 | 18 | -6 |

代码:

```
int dx = abs(x2 - x1);
int dy = abs(y2 - y1);

int x, y;
int p = 2 * dy - dx; //初始决策参数
```

```
for (x; x < x2; x++)
{
    if (p ⩾ 0)
    {
        y++;
        p += 2 * (dy - dx); //这个点画在右上
    }
    else
    {
        p += 2 * dy; //这个点就画在右边
    }

    m_Points.Add(CPoint(x, y)); // 将每个点添加到数组
}
```

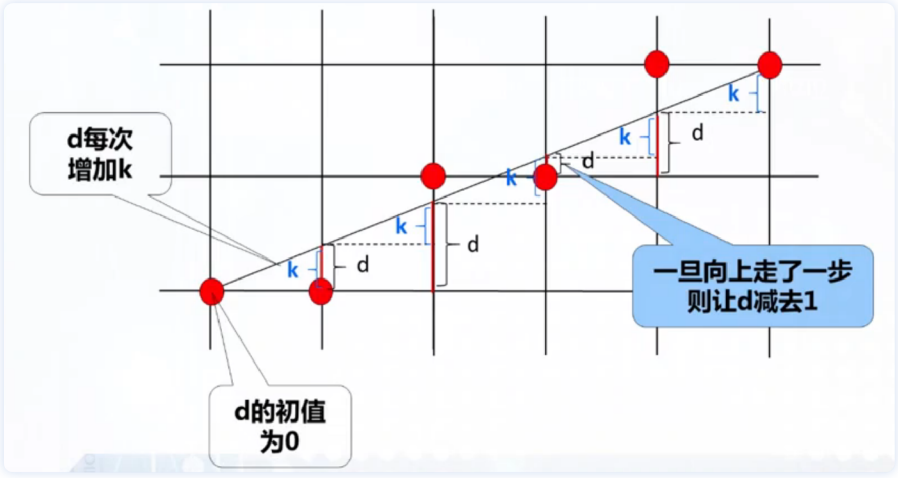
```
//这个适用所有斜率情况
 int x1 = m_PointBegin.x;
 int y1 = m_PointBegin.y;
 int x2 = m_PointEnd.x;
 int y2 = m_PointEnd.y;
 int x = x1, y = y1; //赋初始点
 int dy = y1 - y2, dx = x2 - x1;
 int delta_x = (dx ⩾ 0 ? 1 : (dx = -dx, -1));    //若dx>0则步长为1，否则为-1，同时dx变正
 int delta_y = (dy ⩽ 0 ? 1 : (dy = -dy, -1));    //注意这里dy<0,才是画布中y的增长方向

 dc.SetPixel(x, y, RGB(0,122,204));       //画起始点

 int d, incrE, incrNE;
 if (-dy ⩽ dx)  // 斜率绝对值 ⩽ 1,这里-dy即画布中的dy，画布中y是向下为正所以要取负
 {
     d = 2 * dy + dx;     //初始化判断式d
     incrE = 2 * dy;      //取像素E时判别式增量
     incrNE = 2 * (dy + dx);//NE
     while (x =/= x2)
     {
         if (d < 0)
             y += delta_y, d += incrNE;
         else
             d += incrE;
```

```
                x += delta_x;
                dc.SetPixel(x, y, color);
            }
        }
        else     // 斜率绝对值 > 1, x和y情况互换
        {
            d = 2 * dx + dy;
            incrE = 2 * dx;
            incrNE = 2 * (dy + dx);
            while (y != y2)
            {
                if (d < 0)   //注意d变化情况
                    d += incrE;
                else
                    x += delta_x, d += incrNE;
                y += delta_y;
                dc.SetPixel(x, y, color);
            }
        }
```

# Bresenham直线

## 浮点型

比对y0+k与最近光栅网格点（y或y+1）的距离，决策参数会累积递增（d = d+k），≥1就减掉1




- 把d与0.5的比较优化掉

改进一

d的初值：

$d_0=0$

令e=d-0.5

e的初值：

$e_0=-0.5$

d的变换及如何取点：

d=d+k

$x_{i+1}=x_i+1$

d>0.5 $y_{i+1}=y_i+1$ 此时d=d-1

d≤0.5 $y_{i+1}=y_i$

e的变换及如何取点：

e=e+k

$x_{i+1}=x_i+1$

e>0 $y_{i+1}=y_i+1$ 此时e=e-1

e≤0 $y_{i+1}=y_i$

## 整数型

> Bresenham算法的优点就是减少了浮点数计算

书：

由于 $\Delta x>0$，将式（3-19）的两边同乘以 $2\Delta x$ 并不影响 $e$ 与 0 比较的准确性，因此可令 $e$ 初值为

$$e=2\Delta y-\Delta x \tag{3-20}$$

下面要解决的是 $e'$ 的计算问题。当 $e<0$ 时，选 $P_1$，即 $y$ 方向增量为 0，此时，下一步的误差 $e'$ 计算公式为

$$e'=e+2\Delta y \tag{3-21}$$

而当 $e\geqslant 0$ 时，选 $P_2$，即 $y$ 方向增量为 1，此时，下一步的误差 $e'$ 计算公式为

$$e'=e+2\Delta y-2\Delta x \tag{3-22}$$

前面推导的只是适用于第一八分圆域内直线的 Bresenham 画线算法，下面推导适用于任一八分圆域内直线的 Bresenham 画线算法。

【题3】用Bresenham画线算法扫描转换连接两点$p_0(20,10)$和$p_1(30,18)$的直线段。

解：

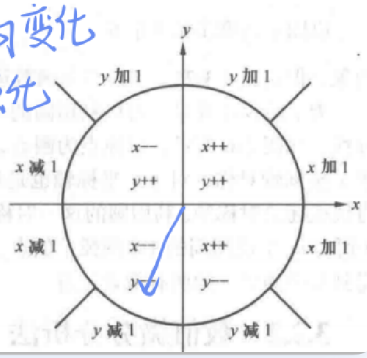
$2\Delta y=16$，$2\Delta y-2\Delta x=-4$，$f_i=2\Delta y-\Delta x=6$

$d_i>0$ $d_i-4$

$d_i<0$ $d_i+16$

| x | y | d |
|---|---|---|
| 20 | 10 | 6 |
| 21 | 11 | 2 |
| 22 | 12 | -2 |
| 23 | 12 | 14 |
| 24 | 13 | 10 |
| 25 | 14 | 6 |
| 26 | 15 | 2 |
| 27 | 16 | -2 |
| 28 | 16 | 14 |
| 29 | 17 | 10 |
| 30 | 18 | 6 |

确定主方向变化和次方向变化



| | 中点画线法 | Bresenham画线法 |
| --- | --- | --- |
| 初始化$d_0$ | $d_0 = \triangle x - 2\triangle y$ | $d_0 = 2\triangle y - \triangle x$ |
| $d_n >= 0$ | $d_{n+1} += -2\triangle y$<br>$x++$ | $d_{n+1} += 2\triangle y - 2\triangle x$,<br>$y_{n+1}$++, $x++$ |
| $d_n < 0$ | $d_{n+1} += 2\triangle x - 2\triangle y$,<br>$y_{n+1}$++, $x++$ | $d_{n+1} += 2\triangle y$<br>$x++$ |

表格里的数学公式表述的不是很严谨，因为懒，但是大意是对的

前面推导的只是适用于第一八分圆域内直线的 Bresenham 画线算法，下面推导适用于任一八分圆域内直线的 Bresenham 画线算法。

首先，要确定最大位移方向。

如图 3-5 所示，当 $|m| = \left|\frac{\Delta y}{\Delta x}\right| < 1$ 时，$x$ 为最大位移方向，$x$ 方向总是加 1 或者减 1；当 $|m| = \left|\frac{\Delta y}{\Delta x}\right| > 1$ 时，$y$ 为最大位移方向，$y$ 方向总是加 1 或减 1。

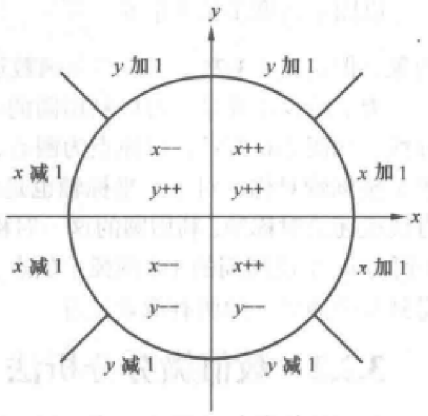


图 3-5 适用于任一八分圆域内直线的 Bresenham 算法的判别条件

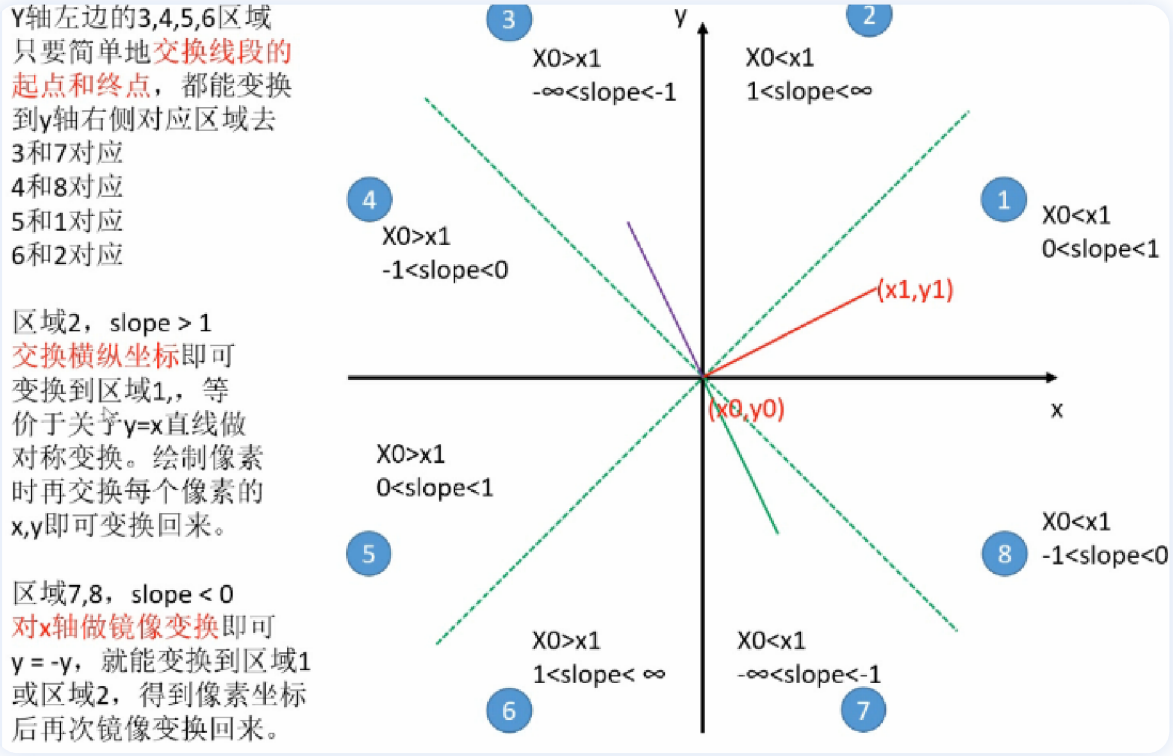
具体是加 1 还是减 1，取决于直线所在的象限。当 $\Delta x \geqslant 0$ 时，$x$ 方向加 1，记为 $S_1=1$；否则 $x$ 方向减 1，记为 $S_1 = -1$。当 $\Delta y \geqslant 0$ 时，$y$ 方向加 1，记为 $S_2 = 1$；否则 $y$ 方向减 1，记为 $S_2 = -1$。

然后，要确定误差 $e$ 的计算方法，并根据误差 $e$ 确定非最大位移方向上的坐标值如何变化。

最大位移方向上的坐标值总是加 1 或减 1，另一个方向上的坐标值是否变化取决于误差 $e$ 的符号。当 $e<0$ 时，如果 $|m|>1$，则 $y=y+S_2$，$e=e+2|\Delta x|$；如果 $|m| \leqslant 1$，则 $x=x+S_1$，$e=e+2|\Delta y|$。当 $e \geqslant 0$ 时，如果 $|m|>1$，则 $x=x+S_1$，$y=y+S_2$，$e=e+2|\Delta x|-2|\Delta y|$；如果 $|m| \leqslant 1$，则 $y=y+S_2$，$x=x+S_1$，$e=e+2|\Delta y|-2|\Delta x|$。

误差 $e$ 的初值由斜率 $m$ 确定，即当 $|m|>1$ 时，$e=2|\Delta x|-2|\Delta y|$，当 $|m| \leqslant 1$ 时，$e=2|\Delta y|-2|\Delta x|$。可见，对于 $|m|>1$ 和 $|m| \leqslant 1$ 两种情况，误差 $e$ 的计算公式中仅是 $\Delta x$ 和 $\Delta y$ 交换的结果。

> 适用于各种斜率的Bresenham算法：
>
> 确定方向变化是加还是减，计算误差时，$\triangle x \triangle y$都加绝对值。斜率绝对值大于1，$\triangle x \triangle y$交换即可




## Bresenham画圆

Bresenham画圆算法在网上有多种不同的递推式，我这里记录的是黄老师PPT和教材上的版本，需要判断三个像素点

和教材的不一样是因为：教材上是使用加一或减一过后的$Y_{n+1}$和$X_{n+1}$来计算$d_{n+1}$

但是我这里的公式是使用没有加减变换过的$Y_n$和$X_n$来计算$d_{n+1}$，其实替换过后是等价的

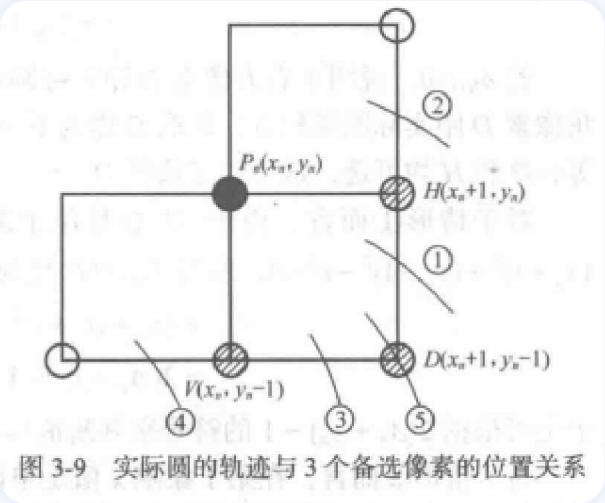


图 3-9　实际圆的轨迹与 3 个备选像素的位置关系

有圆心(xc, yc)

初始：x = 0, y = r, d=2*( 1-r)

```
while (y ≥ 0){ //画第一四分圆
    if(d < 0){ //d小于0,取点(Xn+1, Yn)
        d+= 2x+3 ;
        x++;
    }
    else if (d > 0){ //d大于0, 取点(Xn+1, Yn-1)
        d+= 2(x-y)+6
        x++;
        y--;
    }
    else{ // d等于0, 取点(Xn, Yn-1)
        d+= -2y+3
        y--;
    }

    //给四个个象限的点都上色
    pDC→SetPixel((xc+x),(yc+y),c);//(x,y)
    pDC→SetPixel((xc-x),(yc+y),c);//(-x,y)
    pDC→SetPixel((xc+x),(yc-y),c);//(x,-y)
    pDC→SetPixel((xc-x),(yc-y),c);//(-x,-y)

}
```

| | 中点画圆法 | Bresenham画圆法 |
|---|---|---|
| 初始化 | $x=0, y=r, d_0=(1-r)$ | $x=0, y=r, d_0=2(1-r)$ |
| $d>0$ | $d+=2(x-y)+5,$<br>$y--, x++$ | 如果 $2(d-x)-1<=0$<br>$d+=2(x-y)+6$<br>$x++, y--$<br>否则按d=0处理 |
| $d<0$ | $d+=2x+3$<br>$x++$ | 如果 $2(d+y)-1<=0$<br>$d+=2x+3$<br>$x++$<br>否则按d>0处理 |
| $d=0$ | 同$d>0$ | $d+=-2y+3$<br>$y--$ |

## 中点画圆法

判断d = F(x+1, y-0.5)与0的大小关系

- d >= 0，中点在圆外，y减
- d < 0，中点在圆内，y不减

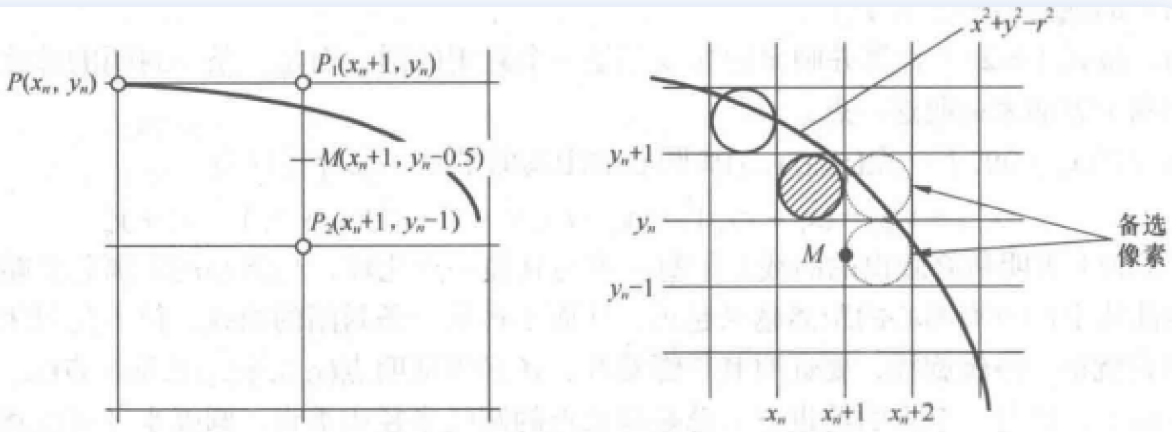



图 3-7　中点画圆算法原理示意图

初始：x = 0, y = r, d = 1-r

表 3-1　以原点为圆心 $r$=10 的第一八分圆的判别式的值和像素位置

| $n$ | $d$ | $2x_n+3$ | $2x_n-2y_n+5$ | $(x_n, y_n)$ |
|---|---|---|---|---|
| 0 | −9 | 3 | — | (0, 10) |
| 1 | −6 | 5 | — | (1, 10) |
| 2 | −1 | 7 | — | (2, 10) |
| 3 | 6 | — | −9 | (3, 10) |
| 4 | −3 | 11 | — | (4, 9) |
| 5 | 8 | — | −3 | (5, 9) |
| 6 | 5 | — | 1 | (6, 8) |
| 7 | 6 | — | — | (7, 7) |

```
while (x < y){
    if(d < 0){ //中点在圆内
        d+= 2x+3;
    }
    else{ //中点在圆外
        d+= 2(x-y)+5;
        y--;
```

```
        }
        x++;
        //给八个象限的点都上色
        pDC->SetPixel((xc+x),(yc+y),c);//(x,y)
        pDC->SetPixel((xc-x),(yc+y),c);//(-x,y)
        pDC->SetPixel((xc+x),(yc-y),c);//(x,-y)
        pDC->SetPixel((xc-x),(yc-y),c);//(-x,-y)
        pDC->SetPixel((xc+y),(yc+x),c);//(y,x)
        pDC->SetPixel((xc-y),(yc+x),c);//(-y,x)
        pDC->SetPixel((xc+y),(yc-x),c);//(y,-x)
        pDC->SetPixel((xc-y),(yc-x),c);//(-y,-x)
    }
```

## （线宽生成）刷子绘制法

之前的讨论的直线和圆弧的生成算法考虑的是**一个像素宽**的直线和圆弧

要获得具有**一定宽度**的线，可以采用：**刷子绘制法**或**实区域填充法**

### 线刷子绘制法

设刷子宽度为w，刷子的w/2处为中点M。M对准直线段的一个端点，向另一端移动

- 如果直线斜率绝对值**小于1**，刷子置为**垂直**方向
- 如果直线斜率绝对值**大于1**，刷子置为**水平**方向

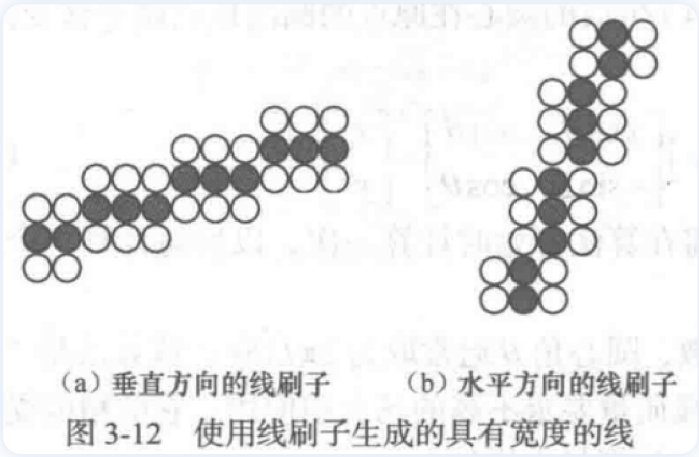



图 3-12　使用线刷子生成的具有宽度的线

### 方形刷子绘制法

指定线宽为w的正方形，让它的中心沿直线或圆弧平行移动

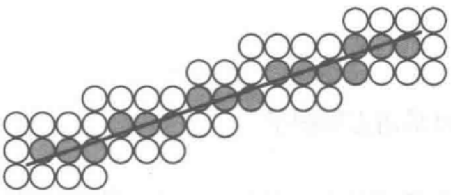


图 3-14　用方形刷子绘制的线条

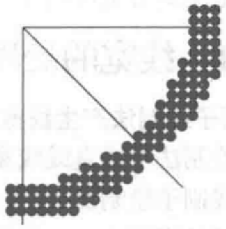


图 3-15　用方形刷子绘制的圆弧

## 实区域填充法

用填充的图形来表示有宽度的直线或圆弧，也就是先用线围出区域再填充

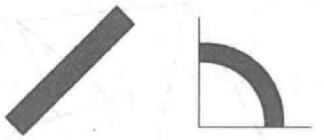


图 3-16　用实区域填充法绘制的线段和圆弧

## 线型的处理

> 怎么生成实线、虚线、点划线?

**位屏蔽器**：一个布尔值的序列，其中每一位的0或1对应这个像素不画还是画

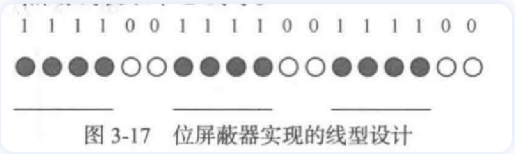



图 3-17　位屏蔽器实现的线型设计

在生成算法中每遍历一个像素就判断：这个像素要不要画

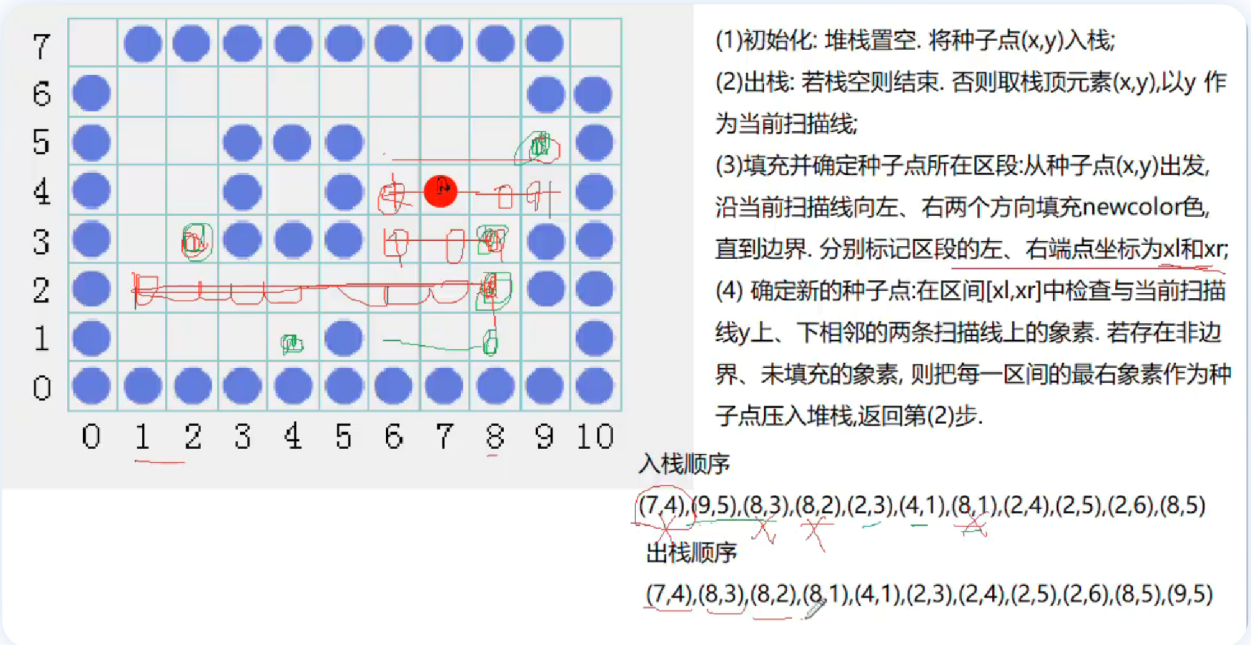
```
if (位串[i % len] == 1) { //len是位屏蔽器的长度，序列中位置i的元素是1就画
    setPixel(x, y, color)
    i++; //每遍历完一个像素，i++
}
```

# 多边形填充算法

## 种子填充法

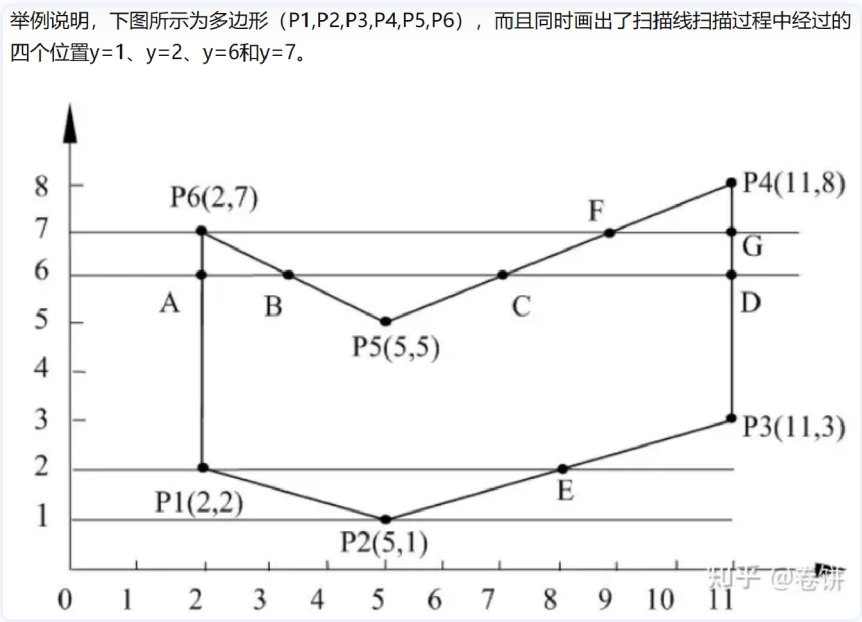
1. 种子入栈
2. 栈顶出栈，对其填色
3. 周围点按照 `左-上-右-下` 的规则入栈(栈中可以有重复元素？)
4. 栈不为空，则重复2，否则结束

## 扫描线种子填充法




## 扫描线填充算法

> 用一条水平方向的扫描线，它从y=0开始，判断与多边形的交点，这些交点把扫描线分成了若干段，我们需要判断哪些段在多边形内部，哪些段在多边形外部，然后把内部的部分着色，完成后，令y=y+1，即扫描线上移一格，重复之前的操作，直到扫描线不再与多边形的任何部分相交




## 边表ET

按边的`yMin`归类，成员有

| x | Δx | ymax | next |
|---|---|---|---|

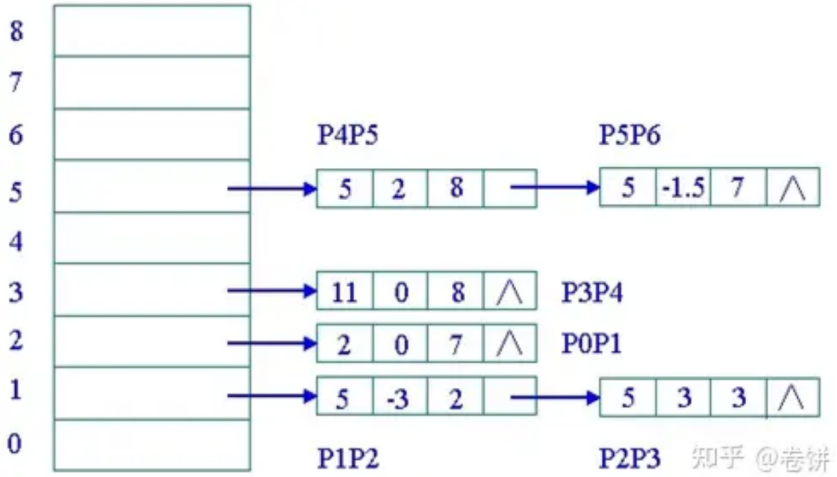
x: 下端点x坐标

Δx: 斜率倒数

ymax: 上端点y坐标

next: 指向下一条边

图示多边形的边表可表示为：



## 活动边表AET

随着扫描线的移动而变化，记录了`扫描线与多边形的相交情况`。若当前的y到达了活动边表内某一边的`ymax`，把该边删除。若当前的y处有新的边信息（查询ET[y]是否为空），把新边加入。然后重新排序，填充。

成员组成和ET相同

| x | Δx | ymax | next |
| --- | --- | --- | --- |

x: 下端点x坐标

Δx: 斜率倒数

ymax: 上端点y坐标

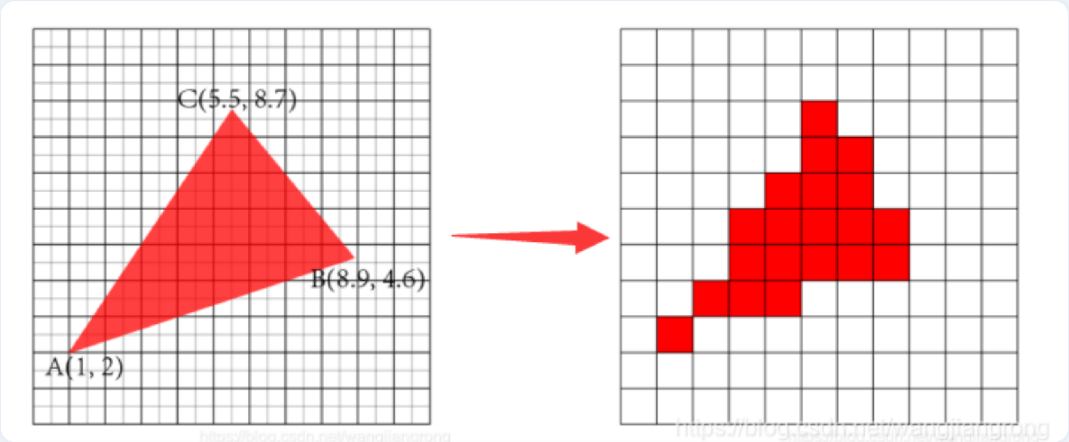
next: 指向下一条边

> 1. 将扫描线初始y坐标设为ET中非空元素的最小序号(yMin)，**从下往上**扫描。
> 2. 将AET初始化为空。
> 3. 循环执行以下步骤，直到ET和AET都变为空。
>    1. 如果 ET[y] 非空，则将其中的所有边取出并插入到AET中，按x（若x相等则按Δx）递增方向排序。
>    2. 删去AET中满足y=ymax的边。
>    3. 若AET非空，将AET中的边按顺序两两**配对并填色**。
>    4. 对于AET中所有边，赋值x = x + Δx。
>    5. y = y + 1，扫描线上移一像素。

示例步骤参考：[扫描线填充算法详解 - 知乎]

# 图形反走样技术

> 走样：根本原因是用离散量（像素点）表示连续量（图形）造成的图形失真。当分辨率又不够高（采样的像素不够多），图形边缘处就会失真




## 像素细分技术

> 高分辨率计算，低分辨率显示

将一个像素均匀分成 `2x2` 或 `3x3` 或 `4x4` 个子像素，假设各个子像素的光亮度值分别为$I_1, I_2, I_3, I_4$ ...

用**算术平均法**（粗暴，总和÷n）或**加权平均法**（每个位置上的子像素有不同的权值）计算出**实际要显示的像素的光亮度值**

## Bresenham反走样技术

对于多边形边上的每一个像素，根据像素与多边形相交的面积值决定像素显示的亮度级别，从而使边界的光亮度能光滑自然地过渡

像素与多边形相交的面积值越大，亮度级别越高

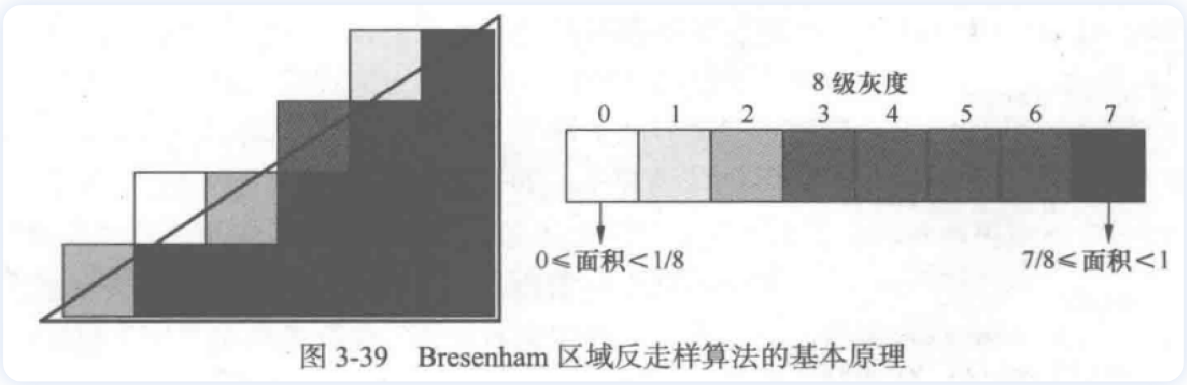



图 3-39　Bresenham 区域反走样算法的基本原理

# 曲线、曲面

## 贝塞尔Bezier曲线(逼近控制点)

1. **曲线的阶次等于控制点的数量减一**。 对于两个点我们能得到一条线性曲线（直线），三个点——一条二阶曲线，四个点——一条三阶曲线

   B(t)就是参数t处对应的点，t=0为起始点，t=1为末端点，其中n = 阶次，每个阶次的基函数不同

n 次贝塞尔曲线的参数方程可以表示为基函数和控制点的线性组合：

$$B(t) = \sum_{i=0}^{n} B_i^n(t) \cdot P_i$$

其中，$P_i$ 是控制点，$B_i^n(t)$ 是基函数，$t$ 的取值范围是 $0 \leq t \leq 1$。

基函数的一般形式如下：

$$B_i^n(t) = C(n,i) \cdot t^i \cdot (1-t)^{n-i}$$

其中，$C(n,i)$ 是组合数，表示从 n 中选择 i 个元素的方式数，$t$ 是参数，$i$ 的取值范围是 $0 \leq i \leq n$，$n$ 是曲线的次数。

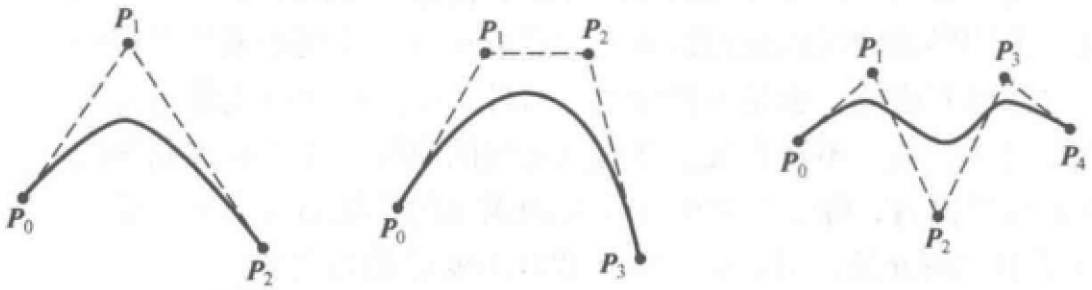



图 4-12　控制点位置与 Bézier 曲线形状的关系

- 二阶Bezier曲线表达式

$$
\begin{aligned}
\boldsymbol{P}(t) &= \sum_{k=0}^{2} \boldsymbol{P}_k \cdot BEZ_{k,2}(t) \\
&= (1-t)^2 \boldsymbol{P}_0 + 2t(1-t)\boldsymbol{P}_1 + t^2 \boldsymbol{P}_2 \qquad 0 \leqslant t \leqslant 1
\end{aligned}
$$

- 二阶Bezier曲线矩阵表达式

$$
\boldsymbol{P}(t) = \begin{bmatrix} \boldsymbol{P}_0 & \boldsymbol{P}_1 & \boldsymbol{P}_2 \end{bmatrix} \begin{bmatrix} 1 & -2 & 1 \\ 0 & 2 & -2 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ t \\ t^2 \end{bmatrix} \qquad 0 \leqslant t \leqslant 1
$$

- 三阶Bezier曲线表达式：

> 等式左边的$B(t)$和$P(t)$一个意思

$$
B(t) = (1-t)^3 \cdot P_0 + 3 \cdot (1-t)^2 \cdot t \cdot P_1 + 3 \cdot (1-t) \cdot t^2 \cdot P_2 + t^3 \cdot P_3
$$

三阶Bezier曲线矩阵表达式：

$$
\boldsymbol{P}(t) = \begin{bmatrix} \boldsymbol{P}_0 & \boldsymbol{P}_1 & \boldsymbol{P}_2 & \boldsymbol{P}_3 \end{bmatrix} \begin{bmatrix} 1 & -3 & 3 & -1 \\ 0 & 3 & -6 & 3 \\ 0 & 0 & 3 & -3 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ t \\ t^2 \\ t^3 \end{bmatrix}
$$

- 四阶Bezier曲线表达式：

$$
[B(t) = (1-t)^4 P_0 + 4(1-t)^3 t P_1 + 6(1-t)^2 t^2 P_2 + 4(1-t) t^3 P_3 + t^4 P_4]
$$

`De Casteljau` 递推算法

# 递推法求 Bezier 曲线上的点

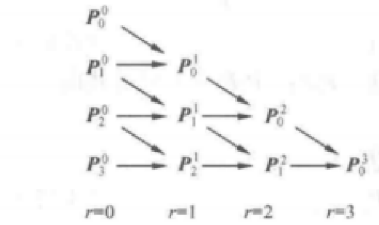



图 4-20　de Casteljau 算法的递推计算过程

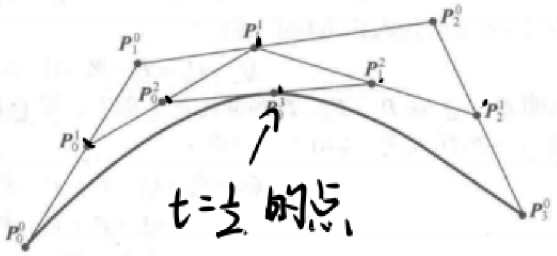
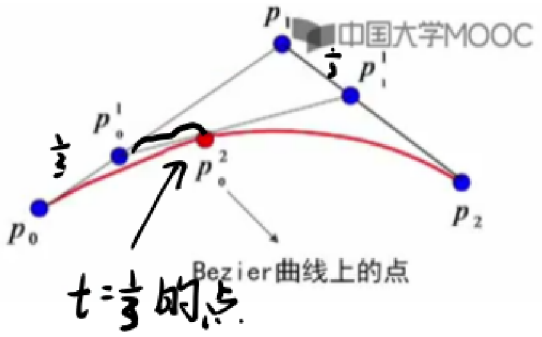



图 4-21　de Casteljau 算法的几何作图过程

$$P_0^1 = (1-t)P_0 + tP_1$$
$$P_1^1 = (1-t)P_1 + tP_2$$
$$P_0^2 = (1-t)P_0^1 + tP_1^1$$




控制点连线，
取每条线上 t 处点，
这些点连线，再取点

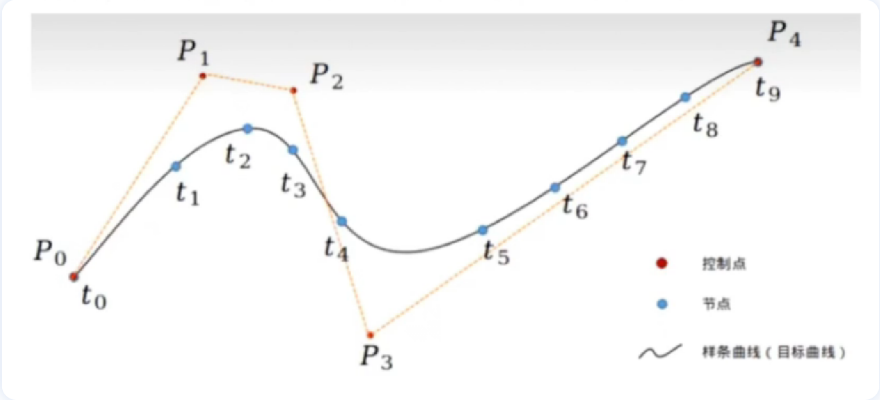
```python
import matplotlib.pyplot as plt

def de_casteljau(points, t):
    """ Recursive implementation of the De Casteljau algorithm. """
    if len(points) == 1:
        return points[0]
    else:
        new_points = []
        for i in range(len(points) - 1):
            x = (1 - t) * points[i][0] + t * points[i + 1][0]
            y = (1 - t) * points[i][1] + t * points[i + 1][1]
            new_points.append((x, y))
        return de_casteljau(new_points, t)

# Define control points
control_points = [(0, 0), (1, 5), (4, 3), (6, 1), (5, -3), (3, -5)]
```

```
# Generate points on the Bezier curve
t_values = np.linspace(0, 1, 100)
curve_points = [de_casteljau(control_points, t) for t in t_values]

# Unpack points for plotting
x_values, y_values = zip(*curve_points)

# Plotting
plt.figure(figsize=(10, 6))
plt.plot(x_values, y_values, label="Bezier Curve")
plt.plot(*zip(*control_points), 'o--', color='grey', label="Control Points")
plt.title("Bezier Curve with De Casteljau Algorithm")
plt.legend()
plt.xlabel("x")
plt.ylabel("y")
plt.grid(True)
plt.show()
```

> Bezier曲线的不足：
>
> 1. Bezier曲线不能对曲线进行**局部控制**，改动一个点，整条线都会受影响
> 2. Bezier曲线要**增加控制点**就必须增加函数的**最高次**，计算复杂
>
> 所以有了B样条曲线，可以局部修改并且多项式次数低，次数与顶点个数没有关系，计算简单

## B样条曲线

- 控制点：控制曲线的点
- 节点：与控制点无关，是人为地将曲线分为若干个部分
  - 若节点有m+1个，即曲线被分为了m份。同时曲线控制点有n+1个，函数的最高次为k，必须满足$m=n+k+1$
  -




- 次（degree）与阶（order）：次是贝塞尔中次的概念，即权重中t的最高幂次。阶=次+1。用k表示次

## 递推公式

- $$B(t) = \sum_{i=0}^{n} W_i P_i == \sum_{i=0}^{n} B_{i,k}(t) P_i$$

  （基函数 $B_{i,k}(t)$，控制点 $P_i$）

  我们需要获得 $W_i$ 即可。$W_i$ 是关于 t 的函数，最高幂次为 k 。B 样条中通常记为 $B_{i,k}(t)$，即表示第 i 点的权重，关于 t 的函数，且最高幂次为 k。而这个权重函数 $B_{i,k}(t)$，在 B 样条里叫做 **k 次 B 样条基函数**

- $B_{i,k}(t)$ 满足如下递推式 (de Boor 递推式)

$$k = 0, \quad B_{i,0}(t) = \begin{cases} 1, & t \in [t_i, t_{i+1}] \\ 0, & \text{Otherwise} \end{cases}$$

$$k > 0, \quad B_{i,k}(t) = \frac{t - t_i}{t_{i+k} - t_i} B_{i,k-1}(t) + \frac{t_{i+k+1} - t}{t_{i+k+1} - t_{i+1}} B_{i+1,k-1}(t)$$

> m+n+1个控制顶点可以生成m+1段n次B样条曲线，每一段由n+1个顶点组成

## 二次B样条曲线的矩阵表示

$$P(t) = \begin{bmatrix} P_0 & P_1 & P_2 \end{bmatrix} \cdot \frac{1}{2} \cdot \begin{bmatrix} 1 & -2 & 1 \\ 1 & 2 & -2 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ t \\ t^2 \end{bmatrix}$$

起点是P0和P1连线的中点，终点是P1和P2的中点

### 三次B样条曲线的矩阵表示

$$P(t)=\begin{bmatrix}P_0 & P_1 & P_2 & P_3\end{bmatrix}\cdot\frac{1}{6}\cdot\begin{bmatrix}1 & -3 & 3 & -1\\ 4 & 0 & -6 & 3\\ 1 & 3 & 3 & -3\\ 0 & 0 & 0 & 1\end{bmatrix}\begin{bmatrix}1\\ t\\ t^2\\ t^3\end{bmatrix}$$

## Bezier曲面

参数多项式曲面是计算机图形学和几何建模中的一个重要概念。它涉及使用多项式函数来定义三维空间中的曲面。这些曲面通过 参数方程 来描述，而不是传统的隐式方程（如 $x^2 + y^2 + z^2 = 1$表示的球面）。参数多项式曲面使得对复杂形状的精确控制和灵活表达成为可能。

参数多项式曲面是通过参数方程定义的。这些方程使用一个或多个参数（通常是 u 和 v）来表示曲面上的点。例如，对于一个曲面 *S*，它的点 *P* 可以表示为：

$$P(u,v) = (x(u,v), y(u,v), z(u,v))$$

其中，$x(u,v), y(u,v),$和 $z(u,v)$ 是多项式函数。

- $x(u,v)$接收两个参数 *u* 和 *v* 作为输入，并输出映射到三维空间中相应点的 X 坐标值

二次Bezier曲面由(2 + 1) * (2 + 1) = 9 个控制顶点组成，三次Bezier曲面由(3 + 1) * (3 + 1) = 16 个控制顶点组成

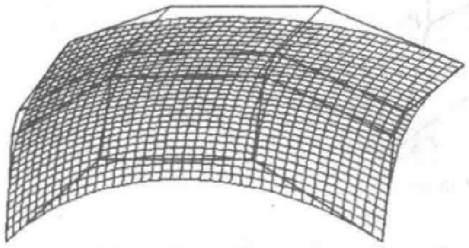


图 4-44　双三次 Bézier 曲面片

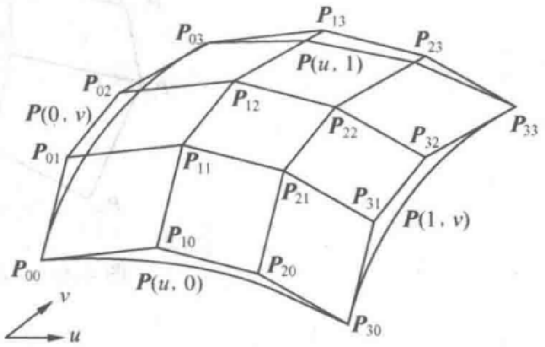



图 4-45　双三次 Bézier 曲面片的特征网格及其控制点

三次Bezier曲面的矩阵表示：

$$\boldsymbol{P}(u,v)=[(1-u)^3 \quad 3u(1-u)^2 \quad 3u^2(1-u) \quad u^3]\begin{bmatrix}\boldsymbol{P}_{00} & \boldsymbol{P}_{01} & \boldsymbol{P}_{02} & \boldsymbol{P}_{03}\\ \boldsymbol{P}_{10} & \boldsymbol{P}_{11} & \boldsymbol{P}_{12} & \boldsymbol{P}_{13}\\ \boldsymbol{P}_{20} & \boldsymbol{P}_{21} & \boldsymbol{P}_{22} & \boldsymbol{P}_{23}\\ \boldsymbol{P}_{30} & \boldsymbol{P}_{31} & \boldsymbol{P}_{32} & \boldsymbol{P}_{33}\end{bmatrix}\begin{bmatrix}(1-v)^3\\ 3v(1-v)^2\\ 3v^2(1-v)\\ v^3\end{bmatrix}$$

$$=[1 \quad u \quad u^2 \quad u^3]\boldsymbol{N}\begin{bmatrix}\boldsymbol{P}_{00} & \boldsymbol{P}_{01} & \boldsymbol{P}_{02} & \boldsymbol{P}_{03}\\ \boldsymbol{P}_{10} & \boldsymbol{P}_{11} & \boldsymbol{P}_{12} & \boldsymbol{P}_{13}\\ \boldsymbol{P}_{20} & \boldsymbol{P}_{21} & \boldsymbol{P}_{22} & \boldsymbol{P}_{23}\\ \boldsymbol{P}_{30} & \boldsymbol{P}_{31} & \boldsymbol{P}_{32} & \boldsymbol{P}_{33}\end{bmatrix}\boldsymbol{N}^{\mathrm{T}}\begin{bmatrix}1\\ v\\ v^2\\ v^3\end{bmatrix}$$

其中：$\boldsymbol{N}=\begin{bmatrix}1 & -3 & 3 & -1\\ 0 & 3 & -6 & 3\\ 0 & 0 & 3 & -3\\ 0 & 0 & 0 & 1\end{bmatrix}$，

## B样条曲面

当 $m=n=3$，B 样条曲面控制网格是一个 4×4 的顶点阵列，如图 4-48 所示，其定义的 B 样条曲面称为双三次 B 样条曲面，用矩阵表示为

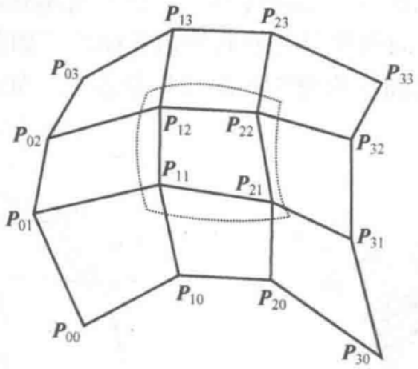



图 4-48　双三次 B 样条曲面片

# 图形变换

## 二维图形变换

### 平移、旋转

例题：

把顶点顶点为A(5, 2), B(1, 1), C(6, 3)的三角形绕C点逆时针旋转45°，写出转换矩阵T和变换后图形顶点的规范化齐次坐标

> 齐次坐标：原本的n维向量用n+1维向量表示，也就是多加一个1
>
> 从普通坐标转换成齐次坐标时：
>
> - 如果(x,y,z)是个点，则变为(x,y,z,1);
> - 如果(x,y,z)是个向量，则变为(x,y,z,0)

由坐标得到变化前的齐次坐标A（一个点为一行）

> 也有 一个点为一列 的写法，教材采用的是行。如果采用列的写法，转换矩阵要转置，运算次序也要改变

三角形点(5, 2) (1, 1) (6, 3)的齐次坐标矩阵

$$\begin{bmatrix} 5 & 2 \\ 1 & 1 \\ 6 & 3 \end{bmatrix} \rightarrow \begin{bmatrix} 5 & 2 & 1 \\ 1 & 1 & 1 \\ 6 & 3 & 1 \end{bmatrix}$$

当我们需要进行**绕任意点旋转或比例变换**时，我们可以把这种情况转换到绕原点的旋转或比例变换，思路如下：

1. 首先将旋转点移动到原点处
2. 执行绕原点（旋转点）的旋转或比例变换
3. 再将旋转点移回到原来的位置

平移的转换矩阵$T_1$：

- $T_X$和$T_y$是平移的的长度，(x, y)移回原点，也就是$T_X = -x$和$T_y = -y$
- **新点 = 旧点 * 转换矩阵** （注意这个顺序！！！）

$$[x' \quad y' \quad 1] = [x \quad y \quad 1] \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ T_x & T_y & 1 \end{bmatrix}$$

旋转的转换矩阵$T_2$：

- 逆时针旋转：θ为正
- 顺时针旋转：θ为负

$$[x' \quad y' \quad 1] = [x \quad y \quad 1] \begin{bmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

平移回去的转换矩阵$T_3$:

$$[x' \quad y' \quad 1] = [x \quad y \quad 1] \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -T_x & -T_y & 1 \end{bmatrix}$$

**总转换矩阵M=**$T_1 * T_2 * T_3$（先转换的矩阵在左)

**变换后的矩阵**$A_1 = A * M$**（注意是齐次矩阵A左乘转换矩阵M）**

## 放缩

注意：这里书上的点齐次坐标以行表示

**3. 相对于任意点（$x_0, y_0$）的比例变换**

相对于任意点$(x_0, y_0)$的比例变换，可以通过以下 3 个步骤得到，当$(x_0, y_0)$为图形重心的坐标时，这种变换实现的就是如图 5-4 所示的相对于重心的比例变换。

**步骤 1**：为了使点$(x_0, y_0)$能平移到原点，先对图形作如下的平移变换，即 $x$ 方向的平移量为$-x_0$，$y$ 方向的平移量为$-y_0$。

$$[x_2 \quad y_2 \quad 1] = [x_1 \quad y_1 \quad 1]\boldsymbol{T}_1 \tag{5-15}$$

式中

$$\boldsymbol{T}_1 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -x_0 & -y_0 & 1 \end{bmatrix}$$

**步骤 2**：将图形相对于原点进行下述比例变换。

$$[x_3 \quad y_3 \quad 1] = [x_2 \quad y_2 \quad 1]\boldsymbol{S} \tag{5-16}$$

式中

$$\boldsymbol{S} = \begin{bmatrix} S_x & 0 & 0 \\ 0 & S_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

**步骤 3**：将图形按照与步骤 1 相反的方向移回$(x_0, y_0)$处，即 $x$ 方向的平移量为 $x_0$，$y$ 方向的平移量为 $y_0$。

$$[x_4 \quad y_4 \quad 1] = [x_3 \quad y_3 \quad 1]\boldsymbol{T}_2 \tag{5-17}$$

式中

$$\boldsymbol{T}_2 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ x_0 & y_0 & 1 \end{bmatrix}$$

将式（5-15）代入式（5-16）中，再将结果代入式（5-17）中，得

$$[x_4 \quad y_4 \quad 1] = [x_1 \quad y_1 \quad 1]\boldsymbol{T} = [x_1 \quad y_1 \quad 1]\boldsymbol{T}_1\boldsymbol{S}\boldsymbol{T}_2 \tag{5-18}$$

## 三维变换

### 平移

1. 平移变换

设空间一点 $P(x,y,z)$ 沿 $x,y,z$ 轴 3 个方向的平移量分别为 $l,m,n$，则

$$\begin{cases} x' = x + l \\ y' = y + m \\ z' = z + n \end{cases}$$

写成矩阵形式为

$$\boldsymbol{T} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ l & m & n & 1 \end{bmatrix}$$

## 比例

2. 比例变换

设空间一点 $P(x,y,z)$ 在 $x,y,z$ 轴 3 个方向的缩放量分别为 $a,e,i$，则以原点为中心的比例变换为

$$\begin{cases} x' = ax \\ y' = ey \\ z' = iz \end{cases}$$

写成矩阵形式为

$$\boldsymbol{T} = \begin{bmatrix} a & 0 & 0 & 0 \\ 0 & e & 0 & 0 \\ 0 & 0 & i & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{5-27}$$

## 绕x轴逆时针旋转

$$\begin{cases} x' = x \\ y' = y\cos\alpha - z\sin\alpha \\ z' = y\sin\alpha + z\cos\alpha \end{cases}$$

$$\boldsymbol{T} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\alpha & \sin\alpha & 0 \\ 0 & -\sin\alpha & \cos\alpha & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

## 绕y轴逆时针旋转

$$\begin{cases} x' = x\cos\beta + z\sin\beta \\ y' = y \\ z' = -x\sin\beta + z\cos\beta \end{cases}$$

$$\boldsymbol{T} = \begin{bmatrix} \cos\beta & 0 & -\sin\beta & 0 \\ 0 & 1 & 0 & 0 \\ \sin\beta & 0 & \cos\beta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

## 绕z轴逆时针旋转

$$\begin{cases} x' = x\cos\gamma - y\sin\gamma \\ y' = x\sin\gamma + y\cos\gamma \\ z' = z \end{cases}$$

$$\boldsymbol{T} = \begin{bmatrix} \cos\gamma & \sin\gamma & 0 & 0 \\ -\sin\gamma & \cos\gamma & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

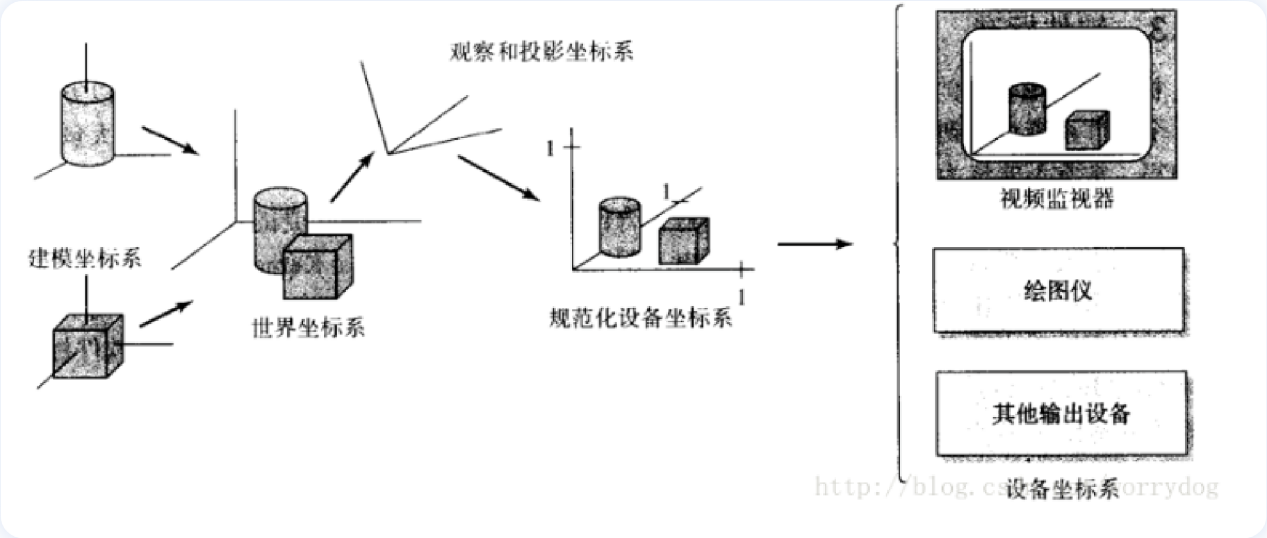
# 坐标系变换

局部坐标系：在描述每一个模型时，采用的是局部坐标系，也就是建模时采用的坐标系，是专用于描述一个图形的，原点通常是模型的中心

世界坐标系：在描述包含了多个模型的场景时，需要使用一个统一的坐标系对多个图形进行定位

摄像机坐标系：通常是以视点的位置为原点，通过用户指定的一个向上的观察向量来定义整个坐标系统，观察坐标系主要用于从观察者的角度对整个世界坐标系内的对象进行重新定位和描述

屏幕坐标系：所有的3D实体最终会显示在一个平面的显示器上，这些实体在屏幕上的位置为二维坐标。屏幕的左上角为坐标原点，水平向右为X轴方向，垂直向下为为Y方向，该坐标系称之为投影坐标系，或者说屏幕坐标系

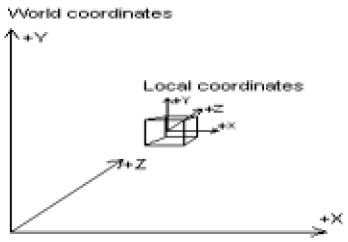


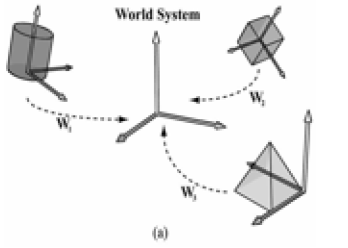

> 要显示一个实体，需要将物体所有的点在 局部坐标系 中的位置转换为在 世界坐标系 中的位置，然后再转换为 摄像机坐标系 中的位置，然后再转换到二维的 屏幕坐标系中 的位置。这一系列的变换都可以通过矩阵变换来完成。

设物体上某点在其局部坐标系中的位置为(x,y,z)，mWorld为将该物体从局部坐标系转换到世界坐标系的转换矩阵，mView为将该物体从世界坐标系转换到摄像机坐标系的转换矩阵，mProj为将该物体从摄像机坐标系转换到屏幕坐标系的转换矩阵，则物体上该点在屏幕上的位置为：

(x,y,z,1)* mWorld* mView* mProj

齐次坐标




# 裁剪算法

## Cohen-Sutherland直线裁剪算法（编码）

Cohen-Sutherland方法：基于编码的裁剪方法

编码：对于任一端点(x,y)，根据其坐标所在的区域，赋予一个4位的二进制码$D_3D_2D_1D_0$。

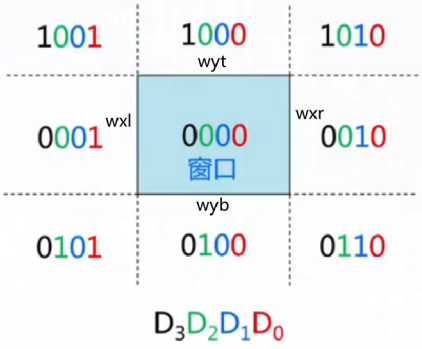
编码规则如下：

◆若x<wxl，则$D_0$=1，否则$D_0$=0；

◆若x>wxr，则$D_1$=1，否则$D_1$=0；

◆若y<wyb，则$D_2$=1，否则$D_2$=0；

◆若y>wyt，则$D_3$=1，否则$D_3$=0。

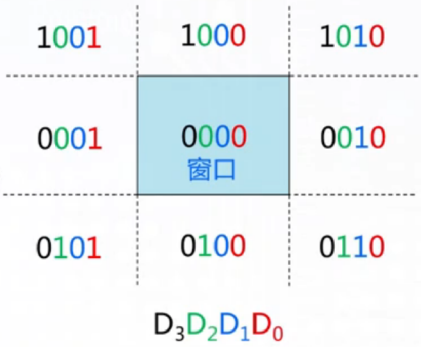


裁剪一条线段时，先求出端点$p_1$和$p_2$的编码code1和code2，然后：

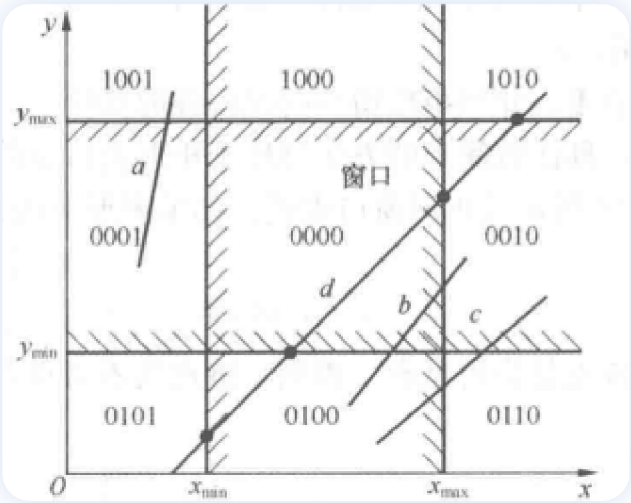
(1)若code1|code2=0，对直线段应**简取**之。
直线全在窗口中

(2)若code1&code2≠0，对直线段可**简弃**之。
直线全不在窗口中

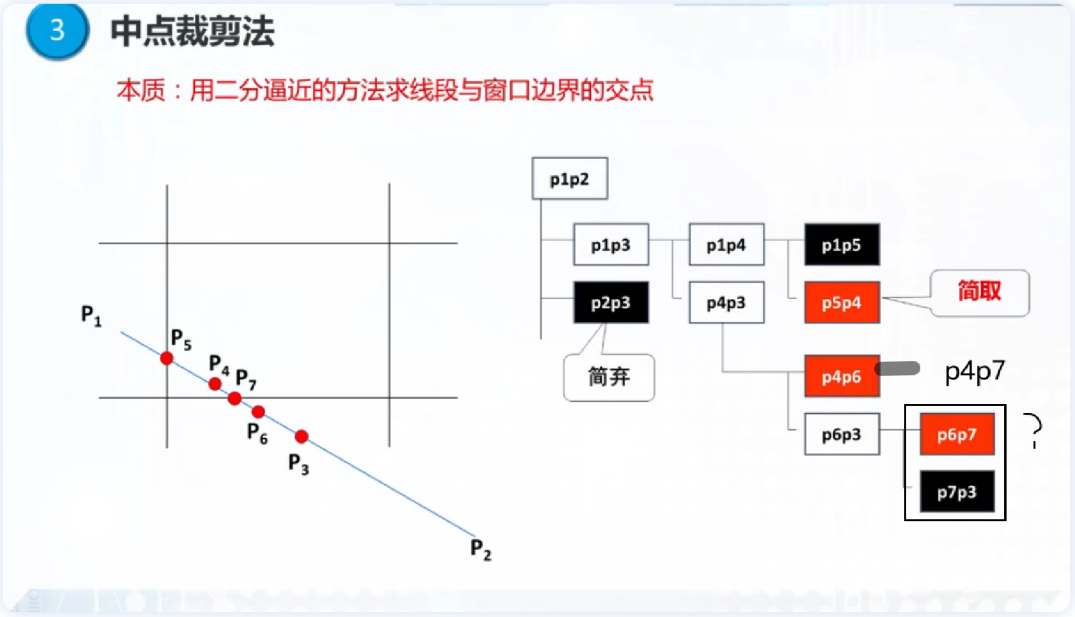
(3)若上述两条件均不成立。则需求出直线段与窗口边界的交点。直线部分在窗口中




到了第三种情况，可能是部分可见或者全不可见




需要求出直线和窗口边界的交点，若交点在边界的延长线上（如上图线段c），则线段位于窗口外；若交点在边界上（如上图线段b），则以一个交点作为分割点，将线段分为两段线段，再分别对两个线段的端点进行编码，进行判断，舍弃完全位于窗口外的线段。




> 中点裁剪法的图示ppt应该有误？（应该取的是p5p4和p4p7）

## Lang-Barsky直线裁剪算法（直线参数方程）

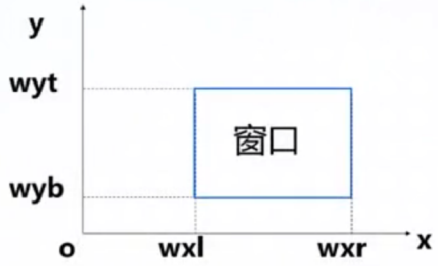
> 教材上并没有使用直线参数方程来表示直线，但思想是一样的，找直线上最晚的进入点和最早的离开点

## 1 Liang-Barsky裁剪的思想

任意直线段I（$X_1,Y_1$）J($X_2,Y_2$)的参数方程：

$$\begin{aligned} x&=x_1+u\cdot(x_2-x_1)\\ y&=y_1+u\cdot(y_2-y_1)\end{aligned} \qquad 0 \le u \le 1$$

给定裁剪窗口：




如果任一点在窗口内则：

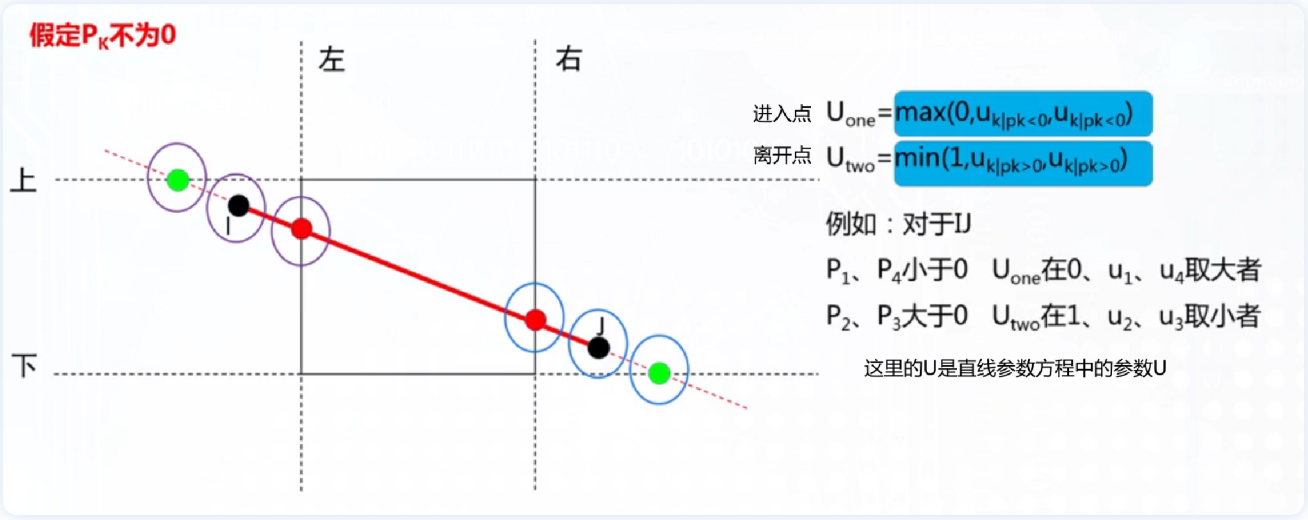
$$\begin{aligned} wxl &\le x_1+u\cdot(x_2-x_1) \le wxr\\ wyb &\le y_1+u\cdot(y_2-y_1) \le wyt\end{aligned}$$

$$\begin{aligned} u\cdot(x_1-x_2) &\le x_1-wxl \quad 左边界\\ u\cdot(x_2-x_1) &\le wxr-x_1 \quad 右边界\\ u\cdot(y_1-y_2) &\le y_1-wyb \quad 下边界\\ u\cdot(y_2-y_1) &\le wyt-y_1 \quad 上边界\end{aligned}$$

令：

$$\begin{aligned} p_1&=x_1-x_2 & q_1&=x_1-wxl\\ p_2&=x_2-x_1 & q_2&=wxr-x_1\\ p_3&=y_1-y_2 & q_3&=y_1-wyb\\ p_4&=y_2-y_1 & q_4&=wyt-y_1\end{aligned}$$

$$u\cdot p_k \le q_k \quad k=1,2,3,4$$




$U_{\text{one}} < U_{\text{two}}$ 才代表直线与窗口有交点
$U_{\text{one}} > U_{\text{two}}$ 代表直线与窗口无交点

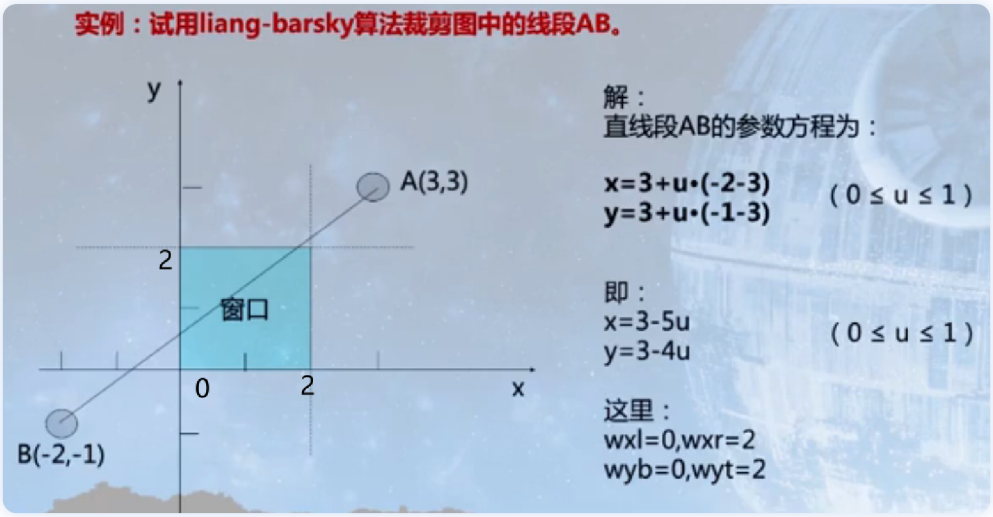
Uone和Utwo就是直线在窗口中起点和终点的参数

Uone ： max(直线和进入的两个边界（如图例中的左、上）的交点的参数, 0)

Utwo ： min(直线和离开的两个边界（如图例中的右、下）的交点的参数, 1)

怎么计算直线和边界的交点参数：

左边界交点： $U_1 = (x_1 - wxl) / (x_1 - x_2)$



因此：

$p_1 = x_1 - x_2 = 5 > 0$　　$q_1 = x_1 - wxl = 3$

$p_2 = x_2 - x_1 = -5 < 0$　　$q_2 = wxr - x_1 = -1$

$p_3 = y_1 - y_2 = 4 > 0$　　$q_3 = y_1 - wyb = 3$

$p_4 = y_2 - y_1 = -4 < 0$　　$q_4 = wyt - y_1 = -1$

可见P均不为0

直线段与窗口边界的交点计算如下：

$u_k \cdot p_k = q_k \quad k=1,2,3,4$

则$u_1=0.6$, $u_2=0.2$, $u_3=0.75$, $u_4=0.25$

由于有：

$p_1$、$p_3$大于0，$p_2$、$p_4$小于0

$U_{one} = \max(0, u_{k|pk<0}, u_{k|pk<0})$

$U_{two} = \min(1, u_{k|pk>0}, u_{k|pk>0})$

因此：

$U_{one} = \max(0, u_2, u_4) = \max(0, 0.2, 0.25) = 0.25$

$U_{two} = \min(1, u_1, u_3) = \min(1, 0.6, 0.75) = 0.6$

可见$U_{one} < U_{two}$，它们分别对应输出直线段的起点和终点

由于有：

$x = 3 - 5u$

$y = 3 - 4u$

因此：

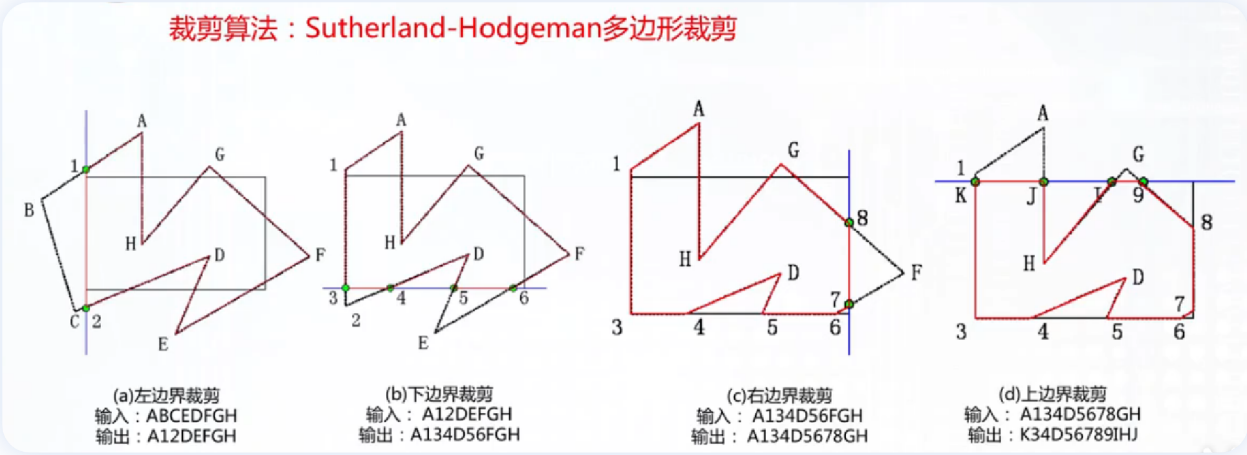
$U_{one}$对应的交点为I（1.75,2）

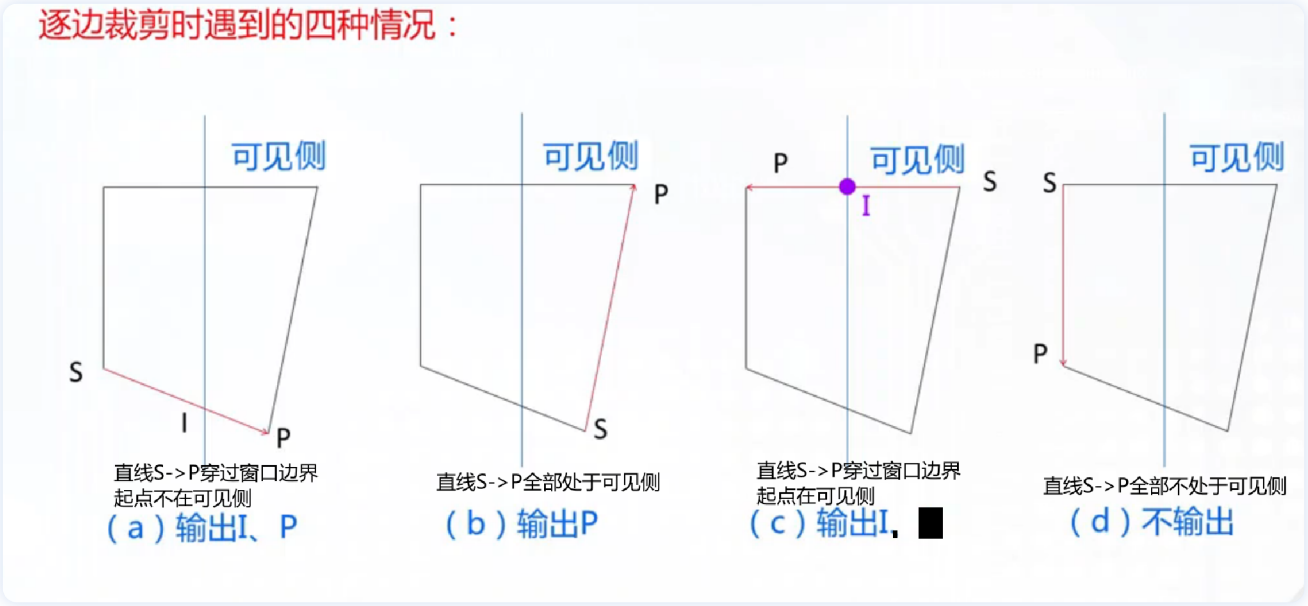
$U_{two}$对应的交点为J(0,0.6)

裁剪后输出线段的端点即为I（1.75,2）和J(0,0.6)，四舍五入后为I（2,2）和J(0,1)。

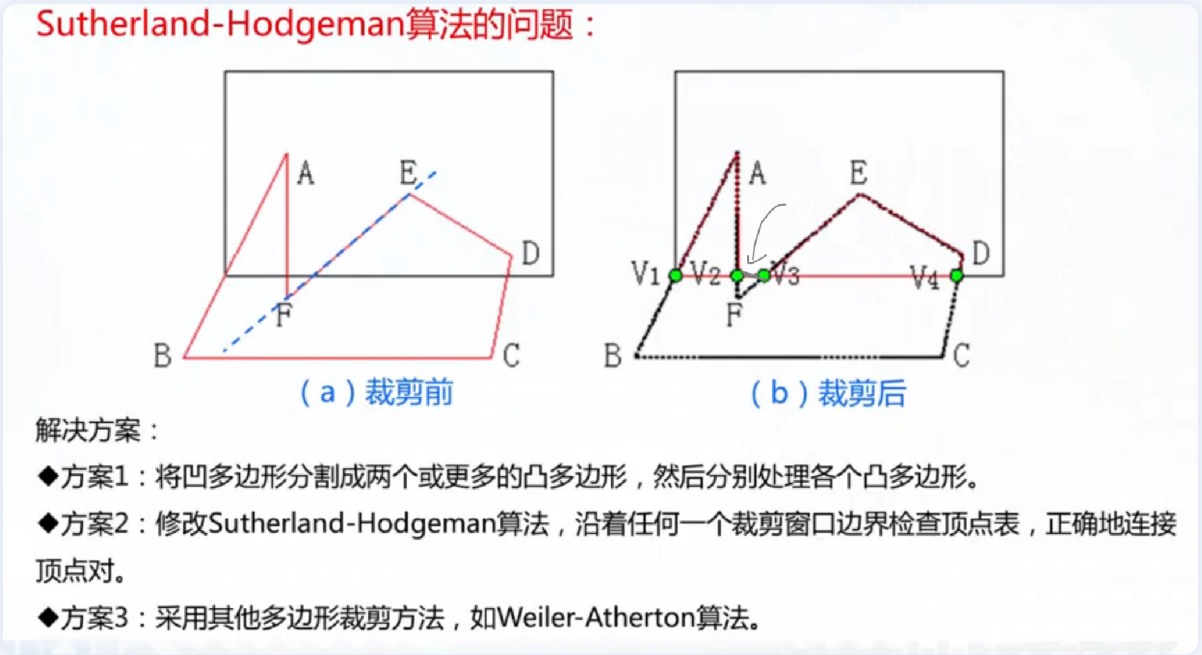
## Sutherland-Hodgeman多边形裁剪算法

将多边形边界作为一个整体，每次用窗口的一条边界对要裁剪的多边形进行裁剪（左边界->下边界->右边界->上边界）






从不可见侧->可见侧才会输出两个点（交点和末端点）

> Sutherland-Hodgman 多边形裁剪算法是有缺陷的，在裁剪凹多边形时会出现多余的边（如下图的 v2→v3 ）
>
> 
> 

1、**如何判断一个点，在裁剪多边形其中一条线的内部还是外部(可见侧还是不可见侧)**

解：设你要拿来判断的裁剪多边形的其中一条线段，它的两个端点坐标分别是**(x1,y1)，(x2,y2)**，你要拿来判断的点的坐标为**(x,y)**，则可以通过下列公式判断：

Given that the line starts from $(x_1, y_1)$ and ends at $(x_2, y_2)$

$$P = (x_2 - x_1)(y - y_1) - (y_2 - y_1)(x - x_1)$$

if P<0, the point is on the right side of the line

P=0, the point is on the line

P>0, the point is on the left side of the line

**如果 P < 0，则这个点位于线的右边**

**如果 P = 0，则这个点位于线上**

**如果 P > 0，则这个点位于线的左边**

2、**如何求得线段与裁剪多边形其中一条线段的交点**

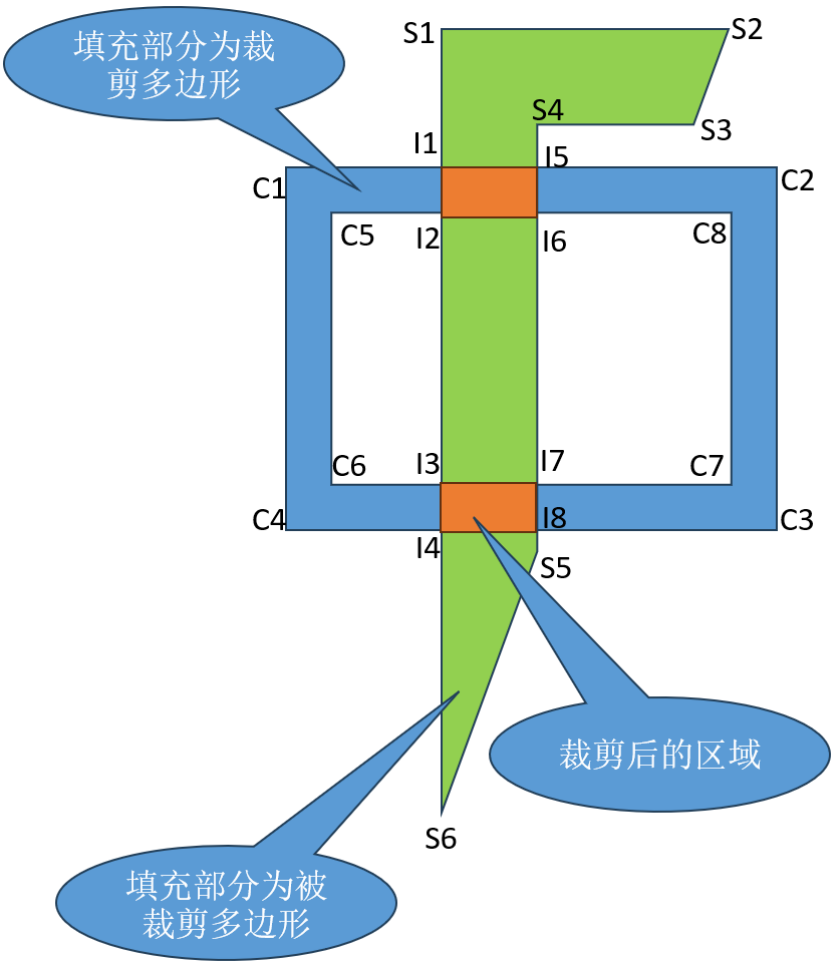
解：设线段两个端点为**(x1,y1)，(x2,y2)**，裁剪多边形其中一点线段两端点为**(x3,y3)，(x4,y4)**，则可以通过以下公式求解：

$$(P_x, P_y) = \left( \frac{(x_1y_2 - y_1x_2)(x_3 - x_4) - (x_1 - x_2)(x_3y_4 - y_3x_4)}{(x_1 - x_2)(y_3 - y_4) - (y_1 - y_2)(x_3 - x_4)}, \frac{(x_1y_2 - y_1x_2)(y_3 - y_4) - (y_1 - y_2)(x_3y_4 - y_3x_4)}{(x_1 - x_2)(y_3 - y_4) - (y_1 - y_2)(x_3 - x_4)} \right)$$

# Weiler-Atherton多边形裁剪算法

裁剪窗口： CP， 被裁剪多边形： SP

进点： 从SP进入CP， 出点： 从CP进入SP




1. 首先将CP和SP的顶点按照“外部边界取顺时针，内部边界取逆时针”的的方式将其构成环形链表

- CP:
  - 外部：C1→C2→C3→C4→C1
  - 内部：C5→C6→C7→C8→C5
- SP: S1→S2→S3→S4→S5→S6→S1

2. 求出CP和SP的交点，分别插入CP和SP的环形链表中（两个链表都要放），并标明 进点 和 出点

- CP:

外：C1→I1(出)→I5(进)→C2→C3→I8(出)->I4(进)→C4→C1

内：C5→C6→I3(出)→I7(进)→C7→C8→I6(出)→I2(进)→C5

- SP:

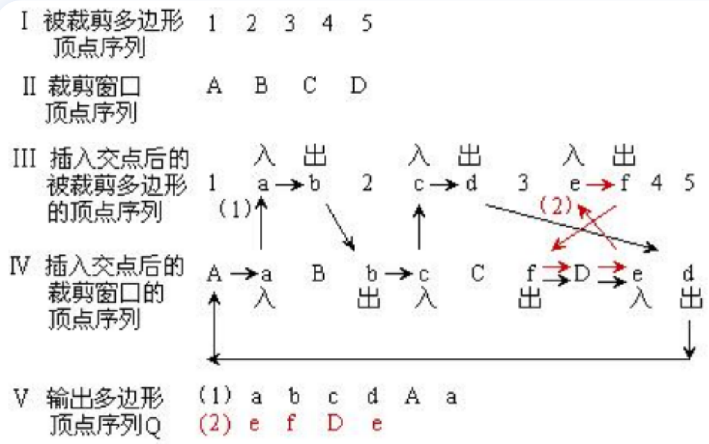
S1→S2→S3→S4→I5(进)→I6(出)→I7(进)->I8(出)→S5→S6→I4(进)→I3(出)→I2(进)->I1(出)→S1

3. 从SP中任一进点P开始，点P送入输出队列，记为本次裁剪的起始点S。若SP中没有进点，算法结束
4. SP中删掉点P的进点标识

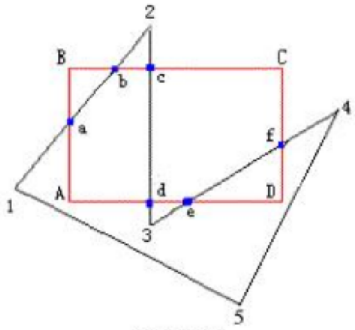
5. 从P点开始，沿着SP的环形链表取下一个节点，记为P，并录入到输出队列Q中， 如果P是出点，则转步骤6，否则转步骤5，也就是继续取SP链表中的下一个节点。
6. 从P点开始，沿着CP的环形链表取下一个节点，记为P，并录入到输出队列Q中， 如果P是出点，则转步骤5，如果P是进点，则判断P是否等于S， 若P≠S， 则转步骤4， 若P=S， 则将队列Q中的点组成的顶点序列输出，然后转步骤3。

总结：从 `SP` 中的一个进点开始遍历并收集点，遇到出点就转到 `CP` ， `CP` 遇到进、出点转回 `SP` 。

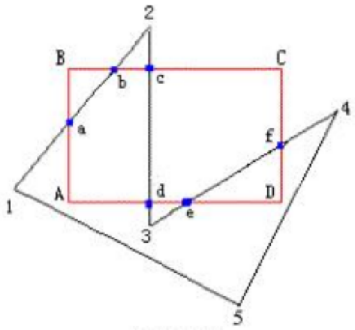
(每在 `CP` 中遍历到进点，要判断是否和起始点S相同，并去掉这个进点在 `SP` 中的进点标记)




图例中W—A算法裁剪结果的生成过程




W-A 算法举例



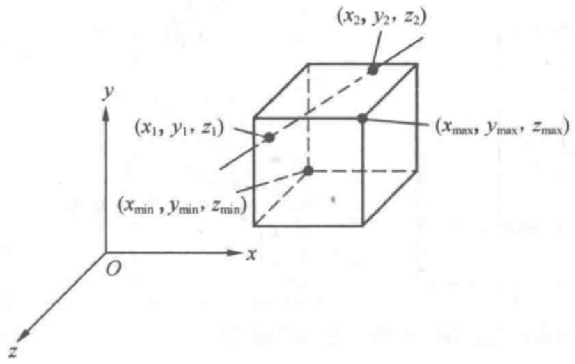

W-A 算法举例

## 长方体裁剪算法

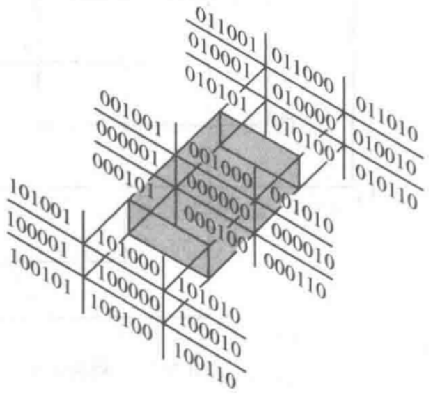
在三维空间中根据裁剪体来裁剪一条直线

是二维编码裁剪直线算法的推广

> 对于直线的的两个端点
>
> 端点位于裁剪体的 左 侧，即 `x<l` ： 编码第0位置为1
>
> 端点位于裁剪体的 右 侧，即 `x>r` ： 编码第1位置为1
>
> 端点位于裁剪体的 下 侧，即 `y<bottom` ： 编码第2位置为1
>
> 端点位于裁剪体的 上 侧，即 `y>top` ： 编码第3位置为1
>
> 端点位于裁剪体的 前 侧，即 `z>front` ： 编码第4位置为1
>
> 端点位于裁剪体的 后 侧，即 `z<rear` ： 编码第5位置为1




（a）平行投影的裁剪体




（b）平行投影的编码方法

若两个端点的编码

- 每一位都是0，则线段全在裁剪体内部，全可见
- 相与，结果为1，则线段全在裁剪体外部，全不可见
- 相与，结果为0，则线段可能部分可见或完全不可见，需要求交点确认

## 视椎体裁剪算法

判断直线端点与视锥体各个面的位置关系

d是投影中心（摄像机）的位置

右平面方程：

$$x-\frac{x_R}{z_Y-d}z-\frac{dx_R}{z_Y-d}=0$$

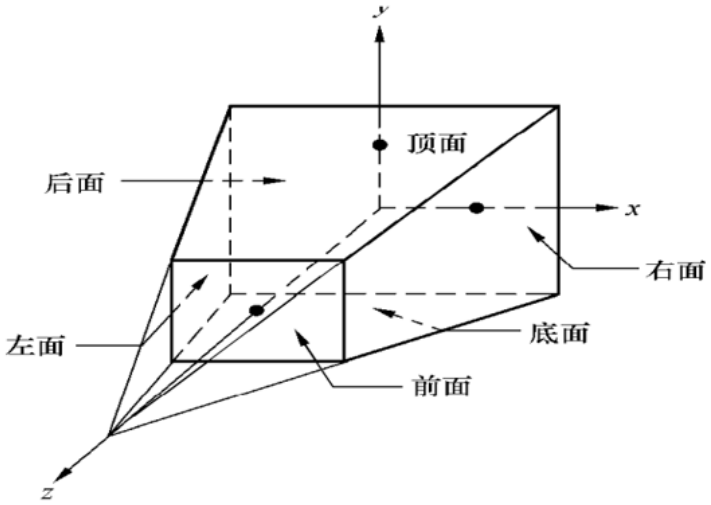
左侧平面的方程：$x-\frac{x_L}{z_Y-d}z-\frac{dx_L}{z_Y-d}=0$

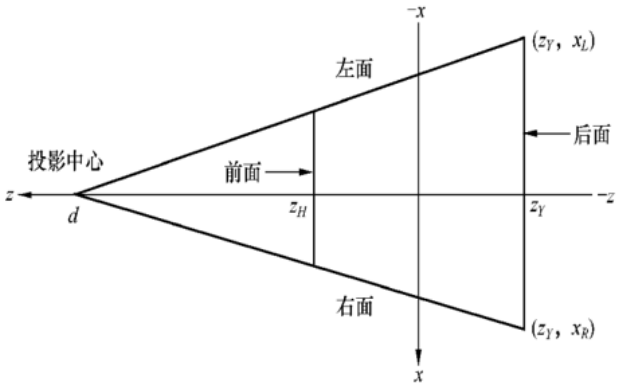
顶部平面的方程：$y-\frac{y_T}{z_Y-d}z-\frac{dy_T}{z_Y-d}=0$

底部平面的方程：$y-\frac{y_B}{z_Y-d}z-\frac{dy_B}{z_Y-d}=0$

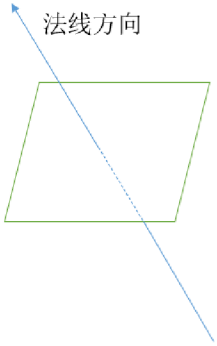
近平面的方程：$z-z_H=0$

远平面的方程：$z-z_Y=0$




因此可以判断直线的端点与视椎体远近平面所组成的区域的关系：




（1）如果对应的方程的值>0，表明在该平面之上。

（2） 如果对应的方程的值=0，表明端点在该平面上。

（3） 如果对应的方程的值<0，表明端点在该平面下。

一般而言，以其外表面的法向方向定义为上，与法向相反的方向为下。

- 对于右平面，若带入点P坐标计算后的方程值大于0，P在其右方（比右平面还右，也就是点P在视锥体外）
- 对于左平面，若带入点P坐标计算后的方程值小于0，P在其左方
- 对于顶面，若带入点P坐标计算后的方程值大于0，P在其上方
- 对于底面，若带入点P坐标计算后的方程值小于0，P在其下方
- 对于前侧面（近平面），若带入点P坐标计算后的方程值大于0，P在其前方
- 对于后侧面（远平面），若带入点P坐标计算后的方程值小于0，P在其后方

# 图形消隐

在图形生成过程中，消去立体实物中被遮蔽的、看不见的部分

## 背面剔除算法

判断一个三维表面是朝向摄像机（正面）还是背向摄像机（背面），是否需要渲染

1. 计算面的法向量$N$与视线向量$V$的夹角$\theta$的余弦，即$\cos\theta = N \cdot V$(点乘)
2. 若$\cos\theta < 0$，即$\theta > 90^\circ$，即表面不可见，不用渲染

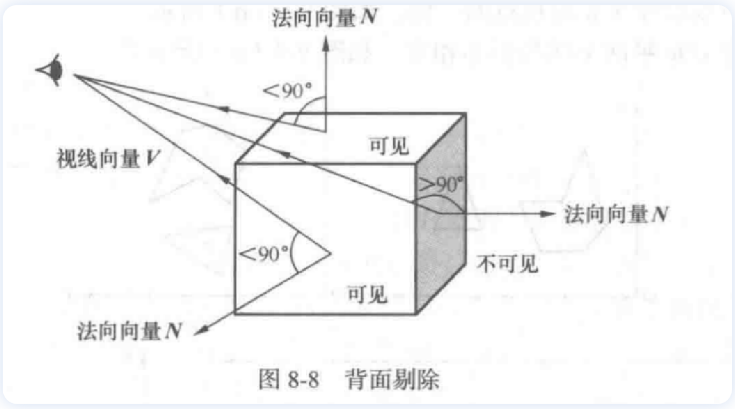


图 8-8　背面剔除

a和b的点积公式为：

$$a \bullet b = a_1 b_1 + a_2 b_2 + \ldots + a_n b_n$$

点乘的几何意义是可以用来表征或计算两个向量之间的夹角，以及在b向量在a向量方向上的投影，有公式：

$$a \bullet b = |a||b|\cos\theta$$

所以这里的法向量向量N和视线向量V需要是单位向量，才能保证算法$\cos\theta = N \cdot V$的正确性

## 画家算法

根据多边形距离视点的距离远近进行排序，生成一个优先队列，离视点远的多边形在队头，近的多边形在队尾。先画离得远的

多边形，再画离得近的，这样后画的多边形就会遮盖掉先画的多边形中不需要显示的部分

## Weiler-Atherton算法

基于WA多边形裁剪算法，总结一下就是 用近的多边形去裁远的多边形

假设现在要生成一个景物实体

> 步骤 1：先进行初步的深度预排序，景物的各表面多边形按各顶点的 z 最小值进行排序，形成景物多边形表。
>
> 步骤 2：以当前具有最大 z 值（即离视点最近）的景物表面作为裁剪多边形 CP 。
>
> 步骤 3：用CP对景物多边形表中排在后面的景物表面进行裁剪，产生内部多边形 Pin 和外部多边形 Pout 。
>
> 步骤 4：由于多边形顶点离视点最近的多边形表面不一定是真正排在最前面的可见面，因此，需比较CP与内部多边形 Pin 的深度，检查 CP是否是真正离视点较近的多边形。 如果是，则CP为可见表面，而位于裁剪多边形 CP之内的多边形 Pin 为当前视点的隐藏面，可以消去该隐藏面；如果不是，则选择 Pin 为新的裁剪多边形，重复步骤 3（如图8-12中的A作为CP去裁剪B生成的Pin就离视点更近）。
>
> 步骤 5：将位于裁剪多边形之外的景物表面 Pout 组成外裁剪结果多边形表，取表中深度最大即排在最前面的表面为裁剪多边形，重复步骤 3，继续对表中其他景物表面进行裁剪。
>
> 步骤 6：上述过程递归进行，直到外裁剪结果多边形表为空时为止

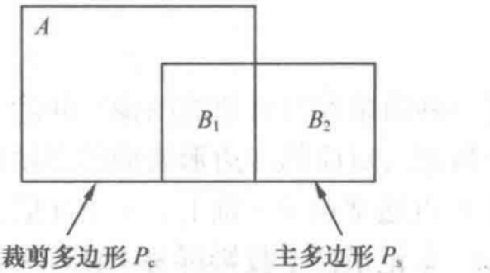



图 8-11　裁剪多边形 $P_c$ 将主多边形 $P_s$ 裁剪为内部多边形 $B_1$ 和外部多边形 $B_2$

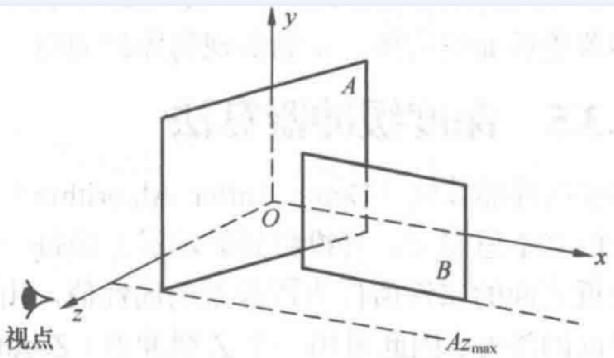



图 8-12　景物表面的深度预排序

## BSP树算法

对景物进行二叉分类，基于和选取平面的前后关系分为front和back，用二叉树存储

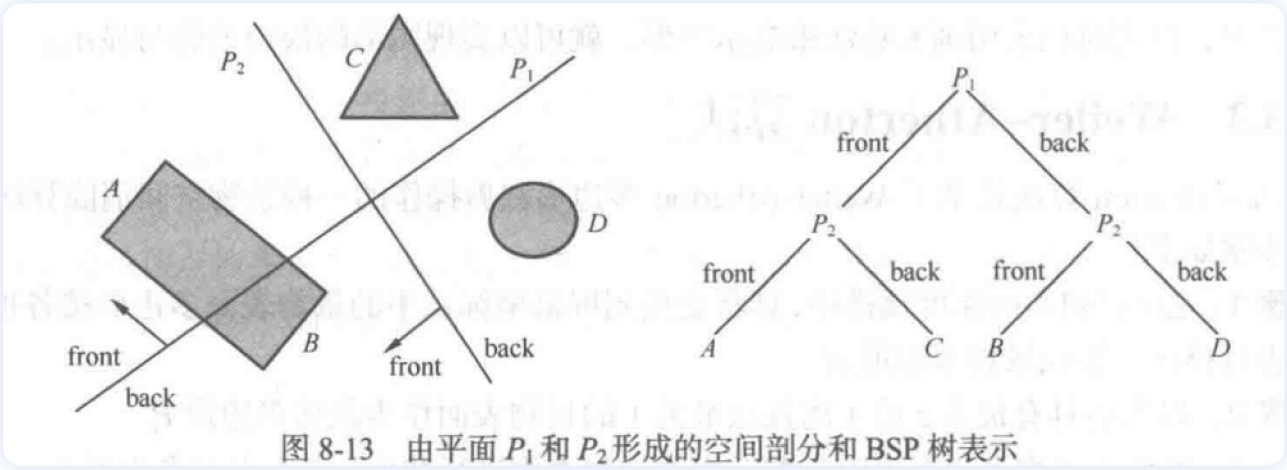


图 8-13　由平面 $P_1$ 和 $P_2$ 形成的空间剖分和 BSP 树表示

> 如上图例
>
> 1. 选空间中一平面P1进行划分，将物体分成两组，一组在平面前，一组在平面后。如果有物体和平面相交，就将其分割成两个物体，分开标识（图中的A和B）。再选一个平面P2对空间进行分割，直到每一个子空间中所含物体数少于给定阈值
> 2. 将分割结果表示为一颗 BSP 树，物体全处于叶子节点，每一个分割平面对应一层，在平面前面的物体作为左分支，平面后面的物体作为右分支
> 3. 基于视点的位置选取，对每一个平面分割出来的子空间进行分类，包含视点的子空间标识为front，另一侧标识为back
> 4. 递归搜索BSP树，优先绘制back子空间中的物体，这样前面的物体就会覆盖掉后面的物体，实现物体的消隐。
>
> *总结*：左右分支是基于物体和平面的前后关系来分的，front和back的标识是基于在选取的视点位置能不能看见物体来分的

## 深度缓冲区算法

基本思想：将投影到显示屏上的每一个像素所对应的多边形的表面深度进行比较，取最近表面的属性值作为该像素的属性值

> 解释：在渲染中，显示屏上一个像素点(x, y)会对应到三维场景中离视点远远近近很多个面上的点(x, y, z)，但这个像素只用渲染出最近的表面属性值，因为更远的面会被最近的面遮盖掉。（左手坐标系，视点一般在z轴上，z坐标值就是各物体离投影平面的距离，最近的点也就是z最小的点）

缓冲区：一般是z缓冲区，存储投影平面上每个像素(x, y)对应的渲染最近的表面的z值

## 算法步骤

```
for（场景中的每一个多边形）
{
    扫描转换该多边形；
    for（多边形所覆盖的每一个像素点(x, y)）
    {
        计算多边形在该像素点的深度值z(x, y)；
        if（ z(x, y)< 深度缓冲器中对应此像素点(x, y)的z值）
        {
            把多边形在(x, y)处的深度值z(x, y)存入深度缓冲器中的(x, y)处；
        }
    }
}
```

## Warnock算法

采取分治的思想，将整个窗口划分为小区域窗口，直到每个小窗口内的情况足够简单，可以直接输出。

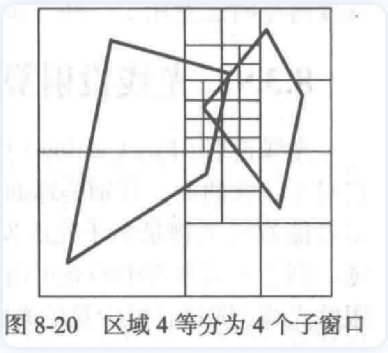


图 8-20　区域 4 等分为 4 个子窗口

怎么判断小窗口是否简单？基于多边形和窗口的关系判断

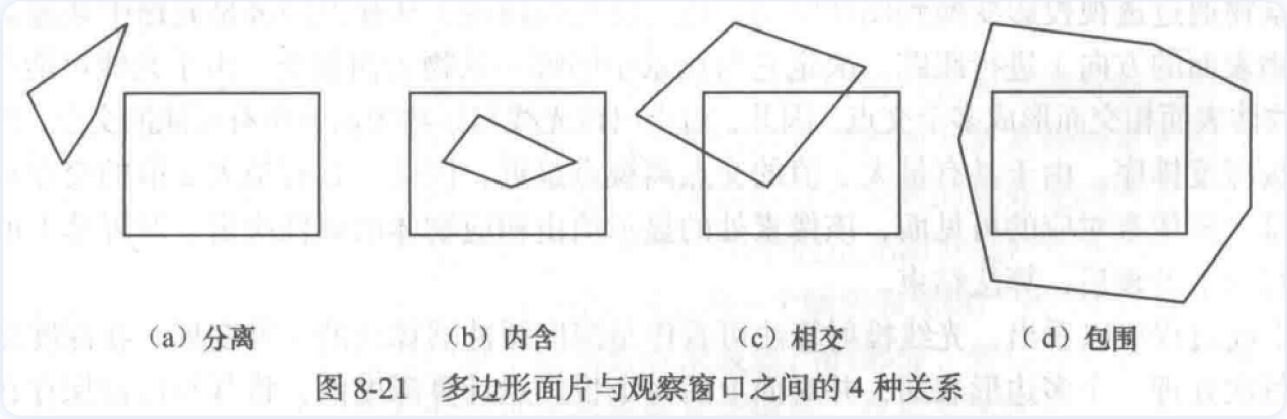


图 8-21　多边形面片与观察窗口之间的 4 种关系

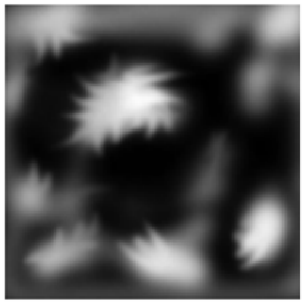
1. 如果窗口中所有多边形都和窗口分离，即窗口为空，就直接显示背景颜色，无需继续分割
2. 如果窗口仅内含一个多边形，即多边形区域显示多边形颜色，非多边形区域显示背景颜色，无需继续分割
3. 如果窗口仅和一个多边形相交，即多边形区域显示多边形颜色，非多边形区域显示背景颜色，无需继续分割
4. 如果窗口仅被一个多边形包围，即整个窗口显示多边形颜色
5. 如果窗口被多个多边形包围，即窗口显示离视点最近的多边形的颜色（*这个情况是消隐的关键*，消去隐藏面）
6. 如果以上情况都不满足，那就继续细分窗口，对小窗口进行判断

# 模型生成

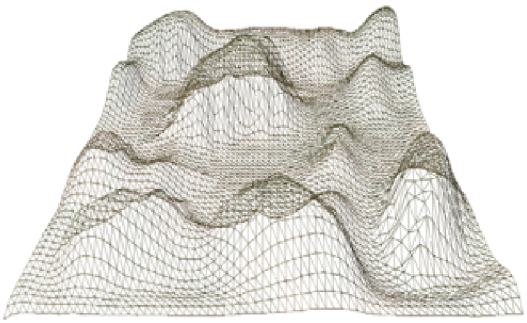
## 地形生成算法

### 根据高度图生成高度

图片的大小与场景的大小基本一致，每个像素的亮度值则为该点的地形高度

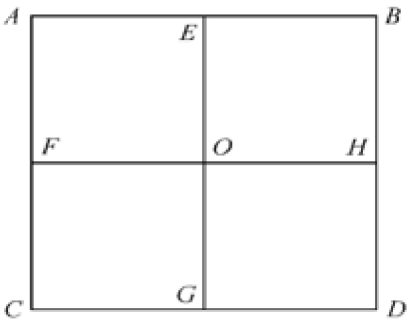


原始的位图



根据位图生成的地形网格

## Diamond-Square算法




1. 给定一个四边形（或三角形），设其四个顶点的坐标为：`A(Xa,Ya,Ha), B(Xb,Yb,Hb), C(Xc,Yc,Hc), D(Xd,Yd,Hd)`，另外定义`初始高度因子h`和`缩放倍数b`
2. 进行迭代。先求出四边形 ABCD 的中心点`O(Xo,Yo)`，并为该点随机生成一个高度值`Ho=(Ha+Hb+Hc+Hd)/4+ h*r`（r 为随机数）于是得到空间中心点`O(Xo,Yo,Ho)`。再分别求出4个边的中点E, F, G, H，中点高度值取为`边的两端点高度的平均值H*r`。
3. 修正高度因子`h=h/b`，用步骤2的方法再分别对这 4 个小正方形进行处理。
4. 重复以上过程直到达到所期望的山形细腻程度。
5. 把生成的所有点连接起来，得到三维山形的轮廓。

## 雨雪模拟算法

采用粒子系统进行模拟，显示的每一个粒子（雨滴、雪花）看成一个具有一定形状的位图

模拟过程包含如下步骤：

> 1. 生成一定数量的初始粒子，对其各个属性进行赋值。
> 2. 更新粒子的运动参数，比如速度，加速度，位置，生命周期等。
> 3. 针对每一个粒子，在粒子对应的位置上绘制位图，并进行位移的映射，背景融合等。
> 4. 删除已经消亡的粒子，释放其占用的资源。

# 基本概念

## 什么是计算机图形学

如何**用计算机生成、处理和显示图形**的一门学科。

## 研究内容

- 硬件部分：输入、输出设备与图形处理设备
- 软件部分：图形生成算法、图形变换算法、图形显示技术

## 图形和图像的区别

- 表示方法不同：图形是由基本几何体（直线，点，圆、曲线、三角形等）构成的实体，同时具有几何属性和视觉属性。图像是由很多像素点构成的点阵信息。
- 生成方法不同：图形是通过计算机算法生成的，而图像是通过照相机，摄像机等扫描设备或图像生成软件制作而成。
- 研究侧重点：图形学研究如何图形学主要研究如何使用计算机表示几何体，构建几何模型、如何通过建立数学模型或者算法把真实的或者想象的物体显示出来。图像处理主要研究如何将一种图像处理成另一种图像，包括图像增强、复原、解析和理解、编码、压缩、匹配，识别等。

## 显卡工作流程

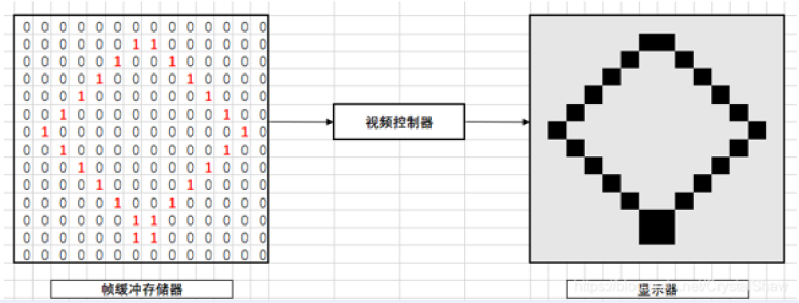
图像或者图形数据在CPU处理后，通过下面四个步骤到达显示器：

1. 通过数据总线将要显示的图形或者图像数据送入GPU（图形处理器)。
2. GPU对送入的数据进行处理然后送入帧缓冲器（或称显存)。

3. 送入帧缓冲器中的数据将被送入视频控制器。视频控制器将根据接口的类型确定处理方式。
4. 视频控制器的输出将送到显示屏

## 显存

显存用来存储屏幕上像素的颜色值，简称帧缓冲器，俗称显存。帧缓冲器中的单元数目与显示器上的像素数目相同，单元与像素一一对应，各单元的数值决定了其对应的像素的颜色。




GPU内存的主要类型及其用途如下：

1. **显存（Video RAM, VRAM）**：这是GPU最重要的内存类型。它专门用于存储渲染图像所需的所有数据，例如纹理、帧缓冲和顶点数据。显存通常是GDDR（Graphics Double Data Rate）类型，如GDDR5或GDDR6，它们提供高带宽以快速处理图形数据。
2. **纹理缓存（Texture Cache）**：这是显存中的一个专门区域，用于存储纹理。纹理是映射到3D对象表面的图像，用于增强视觉效果。纹理缓存有助于快速访问和重用纹理数据，提高渲染效率。
3. **顶点缓存（Vertex Cache）**：这是另一个显存中的专门区域，用于存储3D模型的顶点数据。顶点数据定义了3D对象的形状和结构。顶点缓存加速了顶点数据的处理，使GPU能够快速渲染复杂的3D场景。
4. **常量缓存（Constant Cache）**：用于存储渲染过程中不常变化的数据，如光照参数、变换矩阵等。由于这些数据变化不频繁，将它们存储在常量缓存中可以提高访问效率。
5. **共享内存**
6. **全局内存**

## GPU

GPU是专为执行数学和几何计算而设计的微处理器，能够将复杂的图形转换成显示器能够显示的图像。GPU可通过编程完成所需要的图形变换和计算。

## 显卡分辨率

显卡分辨率 是指 显卡能够处理和输出到显示器的最大像素数量 。分辨率越大，所能显示的图像的像素点就越多，并且能显示更多的细节，当然也就越清晰。

显示器分辨率 ： 这是 显示器能够显示的最大像素数 。显示器分辨率决定了显示器能够展示图像的细节程度。

关键的区别在于，即使显卡支持高分辨率，如果连接的显示器分辨率较低，最终用户看到的图像仍然只能是显示器所能支持的最大分辨率。同样，如果显示器支持更高的分辨率，但显卡不支持，那么图像的分辨率也会被限制在显卡的最大输出分辨率。

## 渲染流水线

渲染流水线的结构如右图所示。主要包括以下几个部分：

（1）输入装配阶段

（2）顶点着色阶段

（3）外壳着色阶段

（4）曲面细分阶段

（5）域着色阶段

（6）几何着色阶段

（7）光栅化阶段

（8）像素着色阶段

## 光照的基本原理

当光照射到物体表面时，物体将吸收一部分光，同时也会反射或透射一部分光。**反射或透射出的光进入人的眼睛所显示出的颜色就是物体的颜色**

## 光照模型、光源

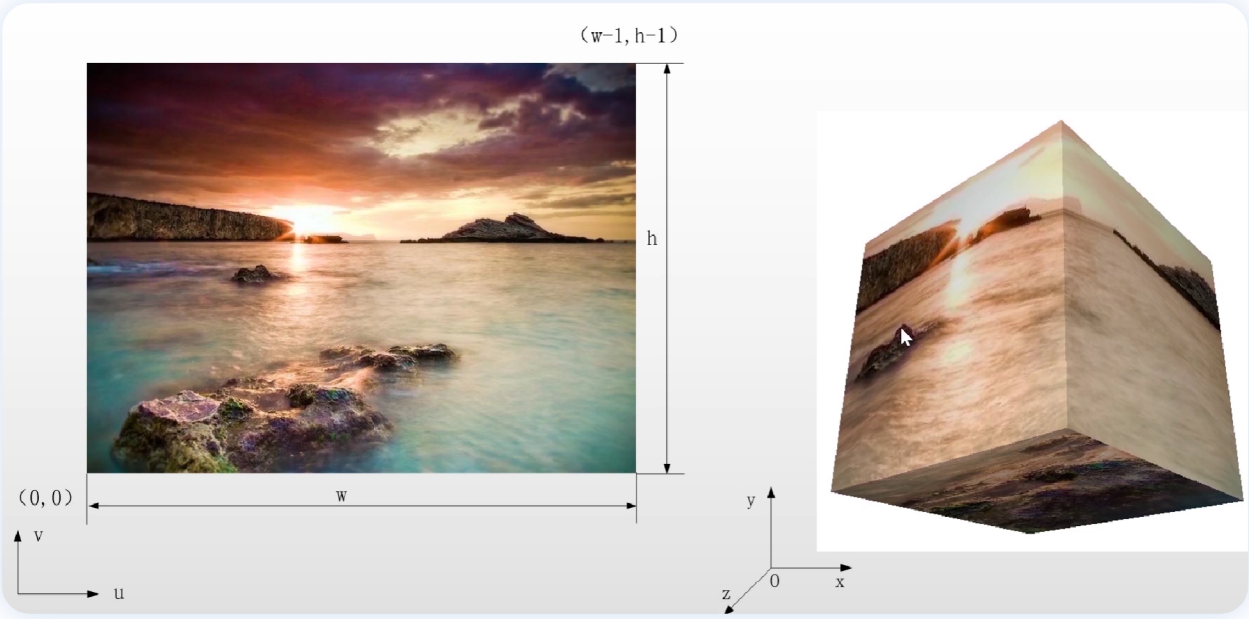
分为：几何光照模型、物理光照模型

分为：点光源、线光源、面光源、体光源

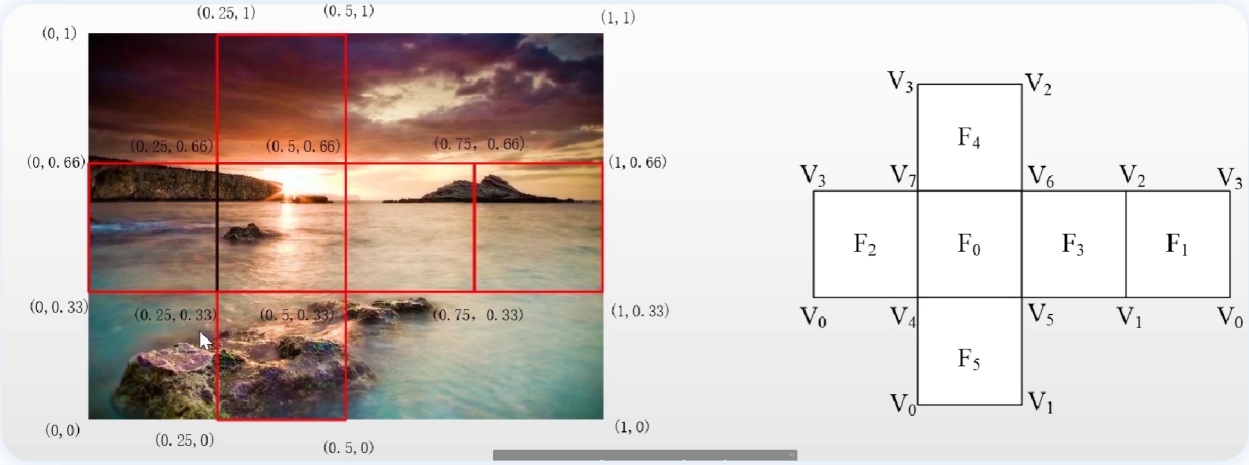
## 纹理

根据纹理的表现形式，可分为颜色纹理，几何纹理（通过修改表面法向矢量表现粗糙程度）和过程纹理

将二维（$u, v \in [0, 1]$）的纹理图片映射到三维(x, y, z)的物体表面上：




立方体图像纹理映射：




## 三维模型顶点的数据结构表示

在计算机图形学中，三维模型顶点的数据结构通常包括以下几个关键部分：

1. **坐标 (Coordinates)**: 这是最基本的部分，包括顶点在三维空间中的位置。通常使用x、y和z坐标来表示。
2. **法线 (Normals)**: 法线是与顶点相关联的向量，它指出了模型的表面在该点的方向。法线用于光照计算和渲染中的其他效果。
3. **切线 (Tangent)**: 切线是实现复杂光照和着色效果（尤其是法线贴图）的重要组成部分。它们使得渲染引擎能够更加精确地模拟光与物体表面的相互作用
4. **纹理坐标 (Texture Coordinates)**: 如果模型将被纹理化，每个顶点还会包含纹理坐标（通常称为UV坐标），这些坐标定义了模型表面上的点与纹理图像上的对应点之间的映射。
5. **颜色 (Color)**: 有时顶点数据还会包含颜色信息，这允许在顶点级别上给模型上色，而不仅仅依赖于纹理。

## 模型的表示方法

物体的表面可以通过很多的三角面构成。因为三角形的三个顶点可以构成一个平面，通过不同大小的空间三角面可以比较准确的模拟出物体的外表面。

需要注意的是，顶点索引的排列顺序决定了三角面法线的方向，在 `Direct3D` 中采用左手定则来确定其法向。比如采用顺时针方式存储这些顶点的索引，则其法向为垂直于该平面向上的方向。

## Direct3D 12图形绘制的基本原理

Direct3D 12绘制3D实体，主要采用流水线绘制方式。实体的显示主要是在GPU内完成的，而CPU主要负责将实体有关的信息准备好，并送入到GPU存储空间中，CPU内存和GPU内存之间的数据传递通过堆（Heap)来完成。

堆分为 `默认堆` 、 `上传堆` 、 `回读堆` 、 `自定义堆`

### 3D实体绘制步骤

1. 设置绘制实体的顶点，并将其传递到默认堆或者上传堆中。（默认堆中的数据在程序运行过程中无法修改其数据，上传堆可以修改）。设置顶点缓冲区视图，对堆中的顶点信息进行描述。
2. 设置绘制实体的三角面的索引，并将其上传到默认堆或者上传堆中。（如果绘制图形的基本单元不是三角面，而是点，线，则没有三角面的索引），设置索引缓冲区视图，对堆中的顶点索引信息进行描述
3. 构建绘制实体必须的纹理。需要将纹理数据传递到上传堆或者默认堆中，然后定义一个描述符堆描述纹理在堆中的地址，并通过根签名绑定对应的纹理寄存器。如前面介绍的内容。
4. 设置绘制实体所需的材质。材质参数以常数的方式上传到上传堆或者默认堆中，设置相应的根签名，存储资源在GPU中的地址。

5. 将所有的资源通过PSO管线状态对象绑定到相应的流水线上。
6. 在每一次绘制时均需要设置每一种资源对应的地址，然后调用相应的绘制函数实现图形的绘制。

# 期末题目回忆

2024年1月

## 简答题：

计算机图形学的软件研究内容、图形和图像的主要区别、渲染流水线

## 计算题

bresenham画线（斜率大于1）、中点画圆（当x是在原点左下方时求其对应y值）、求B样条曲线t=0.1, 0.5, 0.8的点、坐标系转换（三维平移，旋转）、编码直线裁剪（写出线端点的编码和交点的编码）

## 画图题

栅栏填充法、递推法画bezier曲线

## 分析题

纹理坐标映射，把不同数字映射到三维坐标系的立方体的每一个面上

矩形裁剪直线，说明算法过程并计算

## 算法题

代码：计算并收集Bezier曲线上的每个点到数组中

## 笔记作者：swjtu